

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»**

Навчально-науковий інститут атомної та теплової енергетики

Кафедра інженерії програмного забезпечення в енергетиці

ДО ЗАХИСТУ ДОПУЩЕНО

В.о. завідувача кафедри

Олександр КОВАЛЬ

«__» _____ 2026 р.

Дипломна робота

на здобуття ступеня бакалавра

**за освітньо-професійною програмою «Інженерія програмного
забезпечення інтелектуальних кібер-фізичних систем в енергетиці»
спеціальності 121 Інженерія програмного забезпечення**

**на тему: «Розмірно-універсальні моделі резильєнтності кластерних
систем щодо проблеми розщеплення кластера»**

Викона:

студент IV курсу, групи ТВ-23

Сінько Костянтин Дмитрович

(прізвище, ім'я, по батькові)

_____ (підпис)

Керівник:

асист. Макаrchук А.В.

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Рецензент:

_____ (посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Засвідчую, що у цій дипломній
роботі немає запозичень із праць
інших авторів без відповідних
посилань.

Студент _____

(підпис)

Київ – 2026

**Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»**

Навчально-науковий інститут атомної та теплової енергетики

Кафедра інженерії програмного забезпечення в енергетиці

Рівень вищої освіти перший (бакалаврський)

Спеціальність 121 Інженерія програмного забезпечення

Освітньо-професійна програма «Інженерія програмного забезпечення
інтелектуальних кібер-фізичних систем в енергетиці»

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри

_____ Олександр КОВАЛЬ

«___» _____ 2026р.

ЗАВДАННЯ

на дипломну роботу студенту

_____ Сіньку Костянтину Дмитровичу

(прізвище, ім'я, по батькові)

1. Тема роботи: Розмірно-універсальні моделі резильєнтності кластерних систем
щодо проблеми розщеплення кластера

керівник роботи асист. Макарчук А.В.

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «28» травня 2026 р. №1992-с

2. Строк подання студентом роботи «10» червня 2026 р.

3. Вихідні дані до роботи: мова програмування Python, мова програмування
JavaScript, середовище розробки IntelliJ IDEA

4. Зміст (дипломної роботи) пояснювальної записки (перелік завдань, які потрібно
розробити): розробка програмної системи бінарної класифікації конфігурацій
мікросервісних кластерів на основі методів машинного навчання, аналіз
альтернативних підходів до вирішення задачі прогнозування проблеми
розщеплення кластера.

5. Перелік ілюстративного матеріалу: графіки, UML діаграми, Use case діаграми,
скріншоти веб-інтерфейсу

6. Дата видачі завдання «31» жовтня 2025 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів виконання дипломної роботи	Строки виконання етапів роботи	Примітка
1	Отримання завдання	31.10.2025	Виконано
2	Дослідження предметної області	01.11.2025 – 31.11.2025	Виконано
3	Дослідження існуючих рішень	01.12.2025 – 31.12.2025	Виконано
4	Постановка вимог до проєктування системи	01.01.2026 – 31.01.2026	Виконано
5	Розробка програмного продукту	01.02.2026 – 03.05.2026	Виконано
6	Тестування	04.05.2026 – 10.05.2026	Виконано
7	Захист програмного продукту	11.05.2026 – 15.05.2026	Виконано
8	Оформлення дипломної роботи	18.05.2026 – 31.05.2026	Виконано
9	Передзахист	01.06.2026 – 05.06.2026	
10	Захист	15.06.2026 – 19.06.2026	

Студент

(підпис)

Костянтин СІНЬКО

(ім'я, ПРІЗВИЩЕ)

Керівник роботи

(підпис)

Андрій МАКАРЧУК

(ім'я, ПРІЗВИЩЕ)

АНОТАЦІЯ

Структура та обсяг дипломної роботи. Робота містить 77 сторінок, 8 рисунків, 8 таблиць, 3 додатки та 30 посилань.

Метою роботи є розробка та практична реалізація програмної системи бінарної класифікації конфігурацій мікросервісного кластера за станом проблеми розщеплення кластера на основі методів машинного навчання, а також експериментальне дослідження якості реалізованих моделей.

Практичне значення одержаних результатів полягає в розробці повного програмного комплексу із публічним вихідним кодом, що включає шість навчених і відкаліброваних моделей машинного навчання, веб-інтерфейс з інтерактивним графічним редактором топології та документований інструментарій для відтворення експериментів. Розроблена система може бути використана архітекторами розподілених систем для порівняльної оцінки альтернативних топологій кластера на етапі проєктування, а також у дослідницьких цілях як основа для подальшого розвитку методів аналізу структурної надійності розподілених систем.

Апробація результатів дипломної роботи. Основні результати дипломної роботи опубліковано у двох наукових статтях у фаховому журналі «Електронне моделювання» (категорія «Б», ІПМЕ ім. Г.Є. Пухова НАН України), а також у тезах доповіді на науково-практичній конференції «Кібербезпека енергетики», що відбулася 28 травня 2025 року в ІПМЕ ім. Г.Є. Пухова НАН України.

Ключові слова: проблема розщеплення кластера, ПРК, машинне навчання, ансамблеві методи, CatBoost, Random Forest, Gradient Boosting, ізотонічне калібрування, мікросервісна архітектура, розподілені системи.

ABSTRACT

Structure and scope of the thesis. The work contains 77 pages, 8 figures, 8 tables, 3 appendices, and 30 references.

The aim of the work is the development and practical implementation of a software system for binary classification of microservice cluster configurations based on the cluster split-brain problem state, using machine learning methods, as well as an experimental evaluation of the quality of the implemented models.

The practical significance of the obtained results lies in the development of a complete software package with public source code, comprising six trained and calibrated machine learning models, a web interface with an interactive graphical topology editor, and documented tooling for experiment reproducibility. The developed system can be used by distributed systems architects for comparative evaluation of alternative cluster topologies at the design stage, as well as for research purposes as a foundation for further development of structural reliability analysis methods for distributed systems.

Validation of the results. The main results of the thesis have been published in two scientific articles in the professional journal "Electronic Modeling" (Category B, G.E. Pukhov Institute for Modelling in Energy Engineering of the National Academy of Sciences of Ukraine), as well as in the proceedings of the scientific-practical conference "Cybersecurity of Energy Systems", held on May 28, 2025, at the G.E. Pukhov Institute for Modelling in Energy Engineering of the NAS of Ukraine.

Keywords: cluster split-brain problem, SBP, machine learning, ensemble methods, CatBoost, Random Forest, Gradient Boosting, isotonic calibration, microservice architecture, distributed systems.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ	8
ВСТУП.....	9
1 АНАЛІТИЧНИЙ ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ.....	13
1.1 Розподілені системи та мікросервісна архітектура.....	13
1.2 Проблема розщеплення кластера.....	15
1.3 Огляд існуючих підходів до вирішення проблеми	18
1.4 Огляд методів машинного навчання	21
1.5 Постановка задачі	27
Висновки до розділу 1.....	30
2 МЕТОДОЛОГІЯ ДОСЛІДЖЕННЯ.....	31
2.1 Загальна архітектура системи	31
2.2 Формальна постановка задачі	34
2.3 Генерація синтетичних даних	35
2.4 Перетворення даних та формування ознак	37
2.5 Розбиття даних на вибірки.....	39
2.6 Метрики оцінки якості моделей.....	41
Висновки до розділу 2.....	43
3 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ.....	45
3.1 Технологічний стек	45
3.2 Реалізація базових моделей	47
3.2.1 Реалізація CatBoost.....	48
3.2.2 Реалізація Gradient Boosting	50
3.3 Реалізація ансамблевих моделей.....	52
3.3.1 Ансамбль E1_Mixed	54
3.3.2 Ансамбль E2_CB_Bias	54
3.3.3 Ансамбль E3_CB_Hyper	55
3.4 Калібрування ймовірностей.....	56
3.5 Вибір порогу класифікації.....	58
3.6 Веб-інтерфейс системи	60
Висновки до розділу 3.....	63
4 ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ	64

4.1 Методика експериментів	64
4.2 Результати моделей до калібрування	65
4.3 Результати моделей після калібрування.....	68
4.4 Порівняння методів вибору порогу	71
4.5 Порівняльний аналіз моделей	73
4.6 Обмеження дослідження.....	76
Висновки до розділу 4.....	77
ВИСНОВКИ	79
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	82
ДОДАТОК А. Код класу EnsembleModel.....	86
ДОДАТОК Б. Апробація результатів	93
ДОДАТОК В. Презентація.....	96

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

PPK	Проблема розщеплення кластера
AUC	Площа під кривою
CAP	Теорема CAP
CatBoost	Алгоритм градієнтного бустингу
DFS	Алгоритм пошуку у глибину
ECE	Очікувана похибка калібрування
FN	Хибно негативний прогноз
FP	Хибно позитивний прогноз
FPR	Частка хибно позитивних прогнозів
k-NN	Метод k найближчих сусідів
MAE	Середня абсолютна похибка
MCE	Максимальна похибка калібрування
PR-AUC	Площа під PR-кривою
ROC	Робоча характеристика приймача
SPW	Вага позитивного класу при дисбалансі
SVM	Метод опорних векторів
TN	Істинно негативний прогноз
TP	Істинно позитивний прогноз
TPR	Частка істинно позитивних прогнозів

ВСТУП

Сучасні розподілені інформаційні системи характеризуються широким використанням мікросервісної архітектури, яка передбачає розгортання застосунку у вигляді сукупності незалежних домен-орієнтованих сервісів, що взаємодіють через мережеві протоколи. Така архітектура дозволяє забезпечити гнучкість горизонтального масштабування, стійкість до часткових відмов та незалежність технологічного стеку, що зробило її стандартом проєктування для великих критично важливих систем у банківській сфері, телекомунікаціях, електронній комерції та платформах управління інфраструктурою.

Серед критичних станів, які можуть виникати у мікросервісних кластерах, особливе місце займає проблема розщеплення кластера (ПРК). Це такий стан, при якому через порушення мережевої зв'язності кластер фактично поділяється на декілька ізольованих частин, кожна з яких самостійно продовжує обробку транзакцій. Наслідками такого стану є порушення консистентності даних, конфлікти узгодження при подальшому відновленні зв'язку та, у крайніх випадках, повна непрацездатність системи. Запобігання ПРК є однією з ключових задач при проєктуванні розподілених систем. Незважаючи на існування ряду алгоритмічних підходів до зменшення ризиків ПРК, жоден з них не надає механізму кількісної оцінки структурної крихкості конкретної топології кластера на етапі проєктування. Це формує практичну потребу в інструментах аналізу конфігурацій, що дозволяли б архітекторам розподілених систем порівнювати альтернативні топології за схильністю до ПРК ще до розгортання системи у виробничому середовищі.

У контексті українських реалій додатковим джерелом ризиків стають мілітарні загрози та пов'язані з ними перебої у живленні та зв'язку. Значні пошкодження енергетичної інфраструктури призводять до тривалих відключень електропостачання та мережевих розривів між географічно розподіленими частинами інформаційних систем. У таких умовах своєчасна оцінка структурної надійності кластера набуває особливого практичного значення для українських

компаній, що оперують критично важливими інформаційними сервісами. Методи машинного навчання, зокрема ансамблеві методи на основі дерев рішень, демонструють високу ефективність у задачах класифікації даних високої розмірності з дисбалансом класів і є природним інструментарієм для розв'язання поставленої задачі. У роботі досліджується використання цих методів у задачі бінарної класифікації конфігурацій кластерів за станом ПРК на синтетично згенерованих даних.

Метою дипломної роботи є розробка та практична реалізація програмної системи бінарної класифікації конфігурацій мікросервісного кластера за станом ПРК на основі методів машинного навчання, а також експериментальне дослідження якості реалізованих моделей.

Об'єктом дослідження є процес бінарної класифікації конфігурацій мікросервісних кластерів за станом розщеплення на основі топологічних характеристик кластера.

Предметом дослідження є методи машинного навчання на основі ансамблів дерев рішень, процедури калібрування ймовірностей та вибору порогу класифікації, що застосовуються до задачі оцінки ймовірності перебування кластера у стані ПРК.

У роботі використано наступні групи методів. Методи системного аналізу та порівняльного аналізу застосовано при огляді предметної області та оцінці придатності алгоритмів машинного навчання. Методи синтезу використано при проектуванні архітектури програмної системи. Методи статистичного моделювання застосовано при генерації синтетичних навчальних даних за моделлю Ердьош-Реньї. Алгоритмічні методи використано для формування цільових міток. Методи машинного навчання є основним інструментарієм побудови класифікаторів.

Переходячи до практичного значення отриманих результатів, то під час роботи було створено комплекс для оцінки конфігурацій мікросервісних кластерів за ймовірністю ПРК, що включає шість навчених і відкаліброваних моделей

машинного навчання, веб-інтерфейс з інтерактивним графічним редактором топології та документований інструментарій для відтворення експериментів. Розроблена система може бути використана архітекторами розподілених систем для порівняльної оцінки альтернативних топологій кластера на етапі проектування, а також у дослідницьких цілях як основа для подальшого розвитку методів аналізу структурної надійності розподілених систем. Вихідний код системи розміщено у публічному репозиторії, що забезпечує можливість відтворення результатів та подальший розвиток дослідницькою спільнотою.

Під час підготовки тексту дипломної роботи для редагування, перевірки граматики та стилістичного покращення викладу використано допоміжні інструменти на основі штучного інтелекту, а саме великомовну модель Claude, розробка компанії Anthropic та систему машинного перекладу DeepL. Зокрема, DeepL застосовувався для попереднього перекладу окремих фрагментів з англomовних наукових джерел, а Claude для подальшого редагування, узгодження термінології, перевірки граматики та стилістичного шліфування тексту. Усі змістовні рішення, технічну реалізацію системи, проведення експериментів та аналіз отриманих результатів виконано автором особисто. Згадані інструменти застосовувались виключно для роботи з текстовою частиною роботи без впливу на її дослідницький зміст.

У першому розділі проведено аналітичний огляд предметної області: розглянуто особливості розподілених систем та мікросервісної архітектури, проблему ПРК та її наслідки, існуючі алгоритмічні підходи до її запобігання, а також огляд методів машинного навчання, придатних для задач класифікації з дисбалансом класів. На основі огляду сформульовано формальну постановку задачі бінарної класифікації. У другому розділі описано методологію дослідження. Представлено загальну архітектуру системи з діаграмою варіантів використання та компонентною діаграмою, наведено формальне математичне формулювання задачі, описано методику генерації синтетичних даних за моделлю Ердьош-Реньї, обґрунтовано вибір padding-стратегії перетворення матриці суміжності, описано

розбиття даних на чотири незалежні вибірки та зведено перелік метрик оцінки якості. У третьому розділі представлено програмну реалізацію системи. Описано технологічний стек, використання Python 3.10, scikit-learn, CatBoost, Django, NumPy, детально розкрито конфігурацію трьох базових моделей та трьох ансамблевих архітектур, описано процедуру ізотонічного калібрування та три методи пошуку оптимального порогу класифікації. Окремий підрозділ присвячено веб-інтерфейсу системи. У четвертому розділі наведено результати експериментальних досліджень. Представлено метрики моделей до та після калібрування, продемонстровано ефективність калібрування (зниження ESE у понад 200 разів для CatBoost), порівняно три методи вибору порогу, проведено комплексний порівняльний аналіз усіх шести моделей. Окремим підрозділом перелічено обмеження дослідження.

У висновках узагальнено отримані результати, наведено перелік виконаних задач та основні числові показники якості моделей, окреслено обмеження роботи та напрями подальших досліджень.

1 АНАЛІТИЧНИЙ ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Розподілені системи та мікросервісна архітектура

Сучасні інформаційні системи характеризуються постійним збільшенням обсягів даних та кількості користувачів, які взаємодіють з сервісами. Це формує практичну недоцільність обслуговування великих навантажень засобами монолітних архітектур, де весь функціонал застосунку зосереджений в одному виконуваному модулі. Відповіддю на цей виклик стало впровадження принципів розподілених систем, у яких функціональність розбивається на множину незалежно виконуваних компонентів, що взаємодіють між собою через мережеві протоколи.

Розподілені системи мають ряд фундаментальних особливостей, що принципово відрізняють їх від централізованих. Розподілені системи мають відсутність єдиного глобального стану, коли кожен вузол має лише локальне уявлення про загальний стан системи, що оновлюється через обмін повідомленнями з іншими вузлами. Окрім цього вони унеможливають миттєву синхронізацію, бо повідомлення між вузлами передаються із затримками, що залежать від мережевих умов, і у разі географічної віддаленості вузлів затримки можуть бути значними. Існує й можливість часткових відмов. Окремі вузли можуть виходити з ладу або тимчасово втрачати зв'язок із системою, що вимагає відповідної обробки на рівні архітектури застосунку.

Теоретичні обмеження розподілених систем добре відомі у фаховій літературі. Зокрема, теорема CAP стверджує, що у разі поділу мережі розподілена система може забезпечити лише одну з двох властивостей одночасно: консистентність даних (відповідно C) або їхню доступність (позначається як A). Це фундаментальне обмеження формує концептуальні рамки для проектування будь-якої системи, що має працювати у розподіленому середовищі. На практиці розробники обирають один із двох компромісів. CP системи жертвують доступністю заради консистентності, тимчасово блокуючи операції під час

мережевих розривів. Натомість, АР системи зберігають доступність, допускаючи тимчасову неконсистентність.

Серед сучасних підходів до проектування розподілених систем особливе місце займає мікросервісна архітектура [1]. Її ключова ідея полягає у декомпозиції додатку на сукупність незалежних сервісів, кожен з яких відповідає за конкретну функціональну область і має власний життєвий цикл розробки, розгортання та масштабування. Такі сервіси у сукупності й утворюють мікросервісну архітектуру. Така архітектура надає декілька принципових переваг порівняно з монолітними рішеннями. Це дає гнучкість горизонтального масштабування за рахунок можливості додавання нових обчислювальних ресурсів для перевантажених сервісів без необхідності у розширенні всієї системи. Збільшується стійкість до часткових відмов, адже відмова одного сервісу не блокує роботу інших.

Розгортання мікросервісного додатку здійснюється у вигляді кластера, тобто множини обчислювальних вузлів, вони ж ноди, кожен з яких виконує один або декілька мікросервісів. Ноди можуть розташовуватися як в одному центру обробки даних, так і в декількох географічно віддалених локаціях. У контексті критично важливих інформаційних систем, таких як банківські застосунки, телекомунікаційні платформи та системи управління інфраструктурою, розподілення нод між декількома дата-центрами є стандартною практикою забезпечення відмовостійкості. Така стратегія розгортання дозволяє забезпечити роботу системи навіть у разі повного виходу з ладу одного з дата центрів, оскільки навантаження автоматично перенаправляється на доступні ноди в інших локаціях.

Розміри типових кластерів варіюються від декількох вузлів у малих системах до сотень і тисяч вузлів у великих платформах. Для забезпечення консистентності та доступності даних у такому середовищі застосовуються механізми реплікації стану між нодами, протоколи консенсусу та алгоритми виявлення відмов. Зростання кількості вузлів та їхня географічна розподіленість суттєво ускладнює координацію між компонентами і робить актуальною задачу прогнозування критичних станів кластера ще до їхнього настання. Принципово важливою

властивістю мікросервісної архітектури є те, що при виході з ладу одного вузла кластер у цілому продовжує функціонувати. Задачі, що виконувалися на забагованому вузлі, перерозподіляються на інші. Проте така властивість виконується лише за умови, що архітектура передбачає коректну реакцію на часткові відмови і не допускає станів, у яких різні частини кластера приймають суперечливі рішення щодо обробки транзакцій. Саме порушення цієї умови породжує проблему розщеплення кластера, що є предметом дослідження у даній роботі.

Окремо слід зазначити, що у міру збільшення кількості вузлів у кластері зростає не лише потенційна продуктивність системи, але й кількість можливих точок відмови та сценаріїв, у яких система може опинитися у неконсистентному стані. Це робить аналіз структури кластера і його чутливості до різних типів відмов важливим інженерним завданням на етапі проектування. Архітектори розподілених систем потребують інструментів, що дозволяють кількісно оцінювати топології кластерів за критеріями надійності і виявляти конфігурації, схильні до критичних станів, ще до розгортання системи у виробничому середовищі.

1.2 Проблема розщеплення кластера

Проблема розщеплення кластера (далі ПРК) [2], також відома у англomовній літературі як *split brain problem*, є одним із критичних станів розподіленої системи, що виникає внаслідок порушення зв'язності між підмножинами її вузлів. У результаті такого порушення кластер фактично поділяється на декілька ізольованих частин, вони називаються островами, кожен з яких через відсутність комунікації з іншою частиною самостійно вважає себе повноцінним кластером і продовжує обробляти транзакції. Наслідком такої поведінки стає розбіжність стану у двох частинах, що при подальшому відновленні зв'язку призводить до конфліктів даних, втрати інформації або неможливості продовження роботи системи без ручного втручання адміністратора.

Причинами виникнення ПРК є різноманітні відмови мережевої інфраструктури. До найбільш типових сценаріїв належать зокрема повна або часткова втрата зв'язку між дата-центрами, у яких розташовані вузли кластера, а також деградація мережі, за якої окремі повідомлення не доходять у задані часові межі, що інтерпретується протоколом як втрата вузла. Не рідко виникають й програмні збої у мережевому стеку, що блокують прийом або передачу даних.

У контексті українських реалій додатковим джерелом ризиків стають мілітарні загрози та пов'язані з ними перебої у живленні та зв'язку. Згідно з даними Міністерства енергетики України [3], значні пошкодження енергетичних об'єктів призводять до тривалих відключень електропостачання, що безпосередньо впливає на доступність центрів обробки інформації та якість мережевих з'єднань між географічно розподіленими частинами інформаційних систем. У таких умовах виникнення ситуацій, що передують ПРК, стає не виключенням, а постійно діючим ризиковим фактором. Це робить задачу прогнозування критичних станів кластера особливо актуальною для українських компаній, що оперують критично важливими інформаційними сервісами.

Загальна схема сценарію виникнення ПРК наведена на рис. 1.1. На частині а) рисунка показано кластер у нормальному стані, де усі вузли, розподілені між двома дата центрами, мають активні з'єднання як всередині дата центрів, так і між ними. На частині б) показана ситуація після порушення зв'язку між дата центрами. Міждатацентрові з'єднання припиняють функціонувати, і кластер фактично розпадається на два незалежні острови. Кожен острів зберігає внутрішню зв'язність і продовжує вважати себе єдиним функціонуючим кластером.

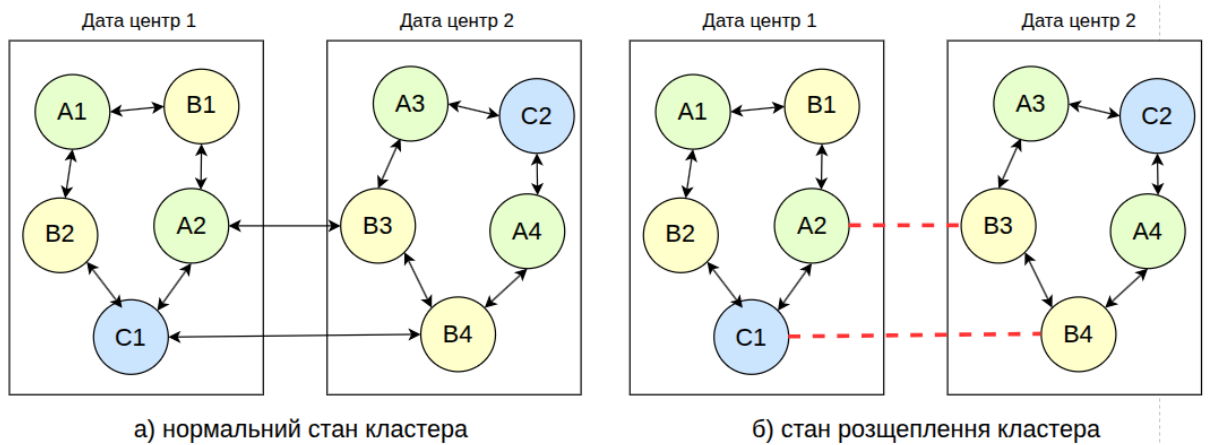


Рисунок 1.1 – Сценарій виникнення ПРК: а) нормальний стан; б) стан розщеплення після порушення зв'язку

Наслідки настання ПРК є серйозними і охоплюють як технічні, так і бізнес аспекти роботи системи. На технічному рівні відбувається порушення консистентності даних. Записи, виконані в одному острові, не відомі іншому острову, а при подальшому відновленні зв'язку постає задача узгодження двох версій станів, що часто є неможливою без втрати частки інформації. На рівні бізнес логіки можливі ситуації подвійних транзакцій, повторного виконання операцій, які мають виконуватися лише одного разу, та видачі суперечливих відповідей різним користувачам системи.

Окремою категорією наслідків є смерть кластера, тобто перехід системи у стан, з якого вона не може самостійно відновитись. У цьому стані жодна з частин кластера не може продовжувати обслуговування транзакцій через втрату кворуму, втрату даних, що зберігалися лише на недоступних вузлах, або через виявлення фундаментальної неконсистентності, яка вимагає ручного втручання адміністратора. Для критично важливих систем смерть кластера може мати каскадні наслідки, оскільки залежні сервіси втрачають доступ до основної функціональності, що призводить до простоїв у роботі цілих категорій бізнес-процесів.

Виявлення стану ПРК у працюючій системі є нетривіальною задачею, оскільки самі вузли не мають повної інформації про стан інших вузлів за межами своєї досяжності. Зазвичай застосовуються комбіновані підходи. З найпопулярніших рішень це періодичні heartbeat повідомлення між вузлами для виявлення тимчасової недоступності, кворумні голосування для прийняття рішень про стан кластера. Кожен з цих механізмів має свої обмеження і не гарантує повного запобігання ПРК у всіх можливих сценаріях.

З огляду на серйозність наслідків ПРК, своєчасне виявлення конфігурацій кластера, схильних до розщеплення, та оцінка структурної крихкості топологій є актуальною науково-технічною задачею. У наступних підрозділах розглядаються відомі підходи до запобігання ПРК на рівні протоколів розподілених систем, а також обґрунтовується вибір методів машинного навчання як інструментарію для оцінки конфігурацій.

1.3 Огляд існуючих підходів до вирішення проблеми

У сучасних розподілених системах існує декілька класів підходів до запобігання ПРК або зменшення її наслідків. Більшість з них спираються на алгоритми консенсусу, кворумні механізми, конфліктно толерантні структури та різноманітні стратегії реплікації стану.

Алгоритми консенсусу є одним із найбільш фундаментальних інструментів забезпечення узгодженості у розподілених системах. Тобто, коли усі вузли кластера прийшли до однакового рішення стосовно певного значення, навіть якщо частина вузлів є недоступною або завагувалися. Серед них найбільш поширеним у промислових застосуваннях є алгоритм Raft [4].

Алгоритм Raft розбиває задачу консенсусу на три основні підзадачі. Він продовдить вибори лідера, створює реплікацію журналу та гарантує забезпечення безпеки. У будь-який момент часу серед вузлів кластера існує не більше одного лідера, що відповідає за обробку всіх записів і реплікацію їх на інші вузли. Якщо

лідер втрачає зв'язок з більшістю вузлів або виходить з ладу, починаються нові вибори. У разі розщеплення кластера алгоритм гарантує, що серед усіх частин лідер обирається лише в тій, де є кворум, а тобто більшість вузлів, а решта частин очікують відновлення зв'язку без обробки записів. Такий підхід знижує ризик неконсистентних даних, проте має й певні обмеження. Меншість, що залишилася без лідера, стає недоступною для запису, що з точки зору доступності системи є непрямую формою її часткової недієздатності.

Принципова обмеженість усіх алгоритмів консенсусу полягає у так званій теоремі FLP, що стверджує неможливість досягнення консенсусу у повністю асинхронній системі з можливістю відмов навіть одного вузла. На практиці це обмеження обходиться через введення часових меж, імовірнісних гарантій або припущень про затримках у мережі, проте у крайніх сценаріях, коли мережа поводить себе непередбачувано, алгоритми консенсусу можуть тимчасово втрачати здатність прийняти рішення.

Конфліктно толерантні структури даних, відомі як CRDT [5], реалізують принципово інший підхід. Замість гарантування єдиного джерела істини у будь-який момент CRDT дозволяють незалежне затвердження змін у різні репліки, причому самі структури даних побудовані таким чином, що при наступному відновленні зв'язку всі зміни автоматично та коректно об'єднуються без втрати інформації. Це досягається за рахунок використання комутативних, асоціативних та ідемпотентних операцій оновлення. Однак CRDT мають обмеження за виразністю. Не всі типи даних і операції можуть бути представлені у такій формі без спотворення семантики. Наприклад, операції, що вимагають глобальної унікальності або строгого порядку, не можуть бути коректно реалізовані через CRDT без додаткових механізмів.

Кворумні системи [6] базуються на ідеї, що для виконання операції читання або запису необхідно отримати підтвердження від певної кількості вузлів кластера, що формують кворум. Поширеним вибором є кворум більшості, що для системи з N вузлів вимагає підтвердження від щонайменше $\lfloor N/2 \rfloor + 1$ вузлів. Така стратегія

гарантує неможливість одночасного існування двох суперечливих рішень у різних частинах кластера за умови, що сума розмірів кворумів читання і запису перевищує загальну кількість вузлів.

Приклад використання кворумних механізмів спостерігається у системі Apache Cassandra [7], що поєднує кворумне підтвердження з декількома додатковими стратегіями забезпечення консистентності. Існує стратегія, яка гарантує, що якщо різні репліки повертають різні значення для одного ключа, система автоматично оновлює застарілі репліки до найновішої версії. Цей механізм та інші механізми разом утворюють комплексний підхід до підтримки консистентності в розподіленому середовищі.

Хмарні провайдери також пропонують вбудовані рішення для зниження впливу ПРК. Зокрема, документація Amazon Web Services [8] описує архітектурні шаблони для побудови масштабованих і відмовостійких застосунків у хмарному середовищі: автоматичне відновлення інфраструктури, балансування навантаження між зонами доступності, реплікацію стану між регіонами. Сервіс має інструменти, що знижують ймовірність ПРК, проте не виключають її повністю і не дозволяють заздалегідь оцінювати ризики конкретної топології кластера.

Окремим аспектом проектування розподілених систем є стратегія реплікації стану. У системах із одним лідером, у англомовній літературі відомо як *single leader*, усі операції запису спочатку проходять через виділений вузол лідер, а потім асинхронно реплікуються на репліки. Такий підхід забезпечує простоту і узгодженість, проте при виході лідера з ладу система потребує часу на обрання нового лідера, що тимчасово блокує запис. У системах із декількома лідерами операції запису можуть прийматися кількома вузлами одночасно, що підвищує доступність, але ускладнює узгодження. Безлідерні системи ж забезпечують максимальну доступність через дозвіл записів на будь-який вузол, проте вимагають додаткових механізмів узгодження стану, таких як CRDT або кворумне читання, що не завжди є найоптимальнішим рішенням ситуації.

Аналіз перелічених підходів показує, що жоден з них не розв'язує проблему ПРК повністю. Усі вони пропонують компроміси між узгодженістю, доступністю та толерантністю до розділень мережі, що відповідає теоремі CAP. Крім того, усі ці рішення працюють на рівні протоколу і не дозволяють оцінювати конфігурацію кластера на етапі її проектування. Це формує практичну потребу у методах, які дозволяли б архітекторам і інженерам кластерних систем кількісно оцінювати схильність конкретної топології до ПРК ще до її розгортання у виробничому середовищі. Саме у цьому контексті актуальним є застосування методів машинного навчання як інструментарію аналізу конфігурацій.

1.4 Огляд методів машинного навчання

Задача оцінки конфігурацій кластера за станом ПРК належить до класу задач бінарної класифікації, для розв'язання яких розроблено широкий спектр методів машинного навчання. Особливістю даної задачі є значний дисбаланс класів. При випадковій генерації топологій частка конфігурацій, що відповідають стану ПРК, є невеликою порівняно з частками нормальних конфігурацій. Це накладає додаткові вимоги до обраних методів. Вони мають бути ефективними у виявленні рідкісних класів і коректно працювати з нерівномірно розподіленими навчальними вибірками.

Перед детальним розглядом окремих методів варто зазначити загальні підходи до роботи з дисбалансом класів. Одним з підходів є збагачення тренувальної вибірки прикладами рідкісного класу через штучну генерацію або зменшення представництва частого класу, а також синтетичне доповнення міноритарного класу за допомогою алгоритмів типу SMOTE. Окремим важливим аспектом є калібрування ймовірностей після навчання, оскільки моделі, навчені на штучно збагачених вибірках, систематично переоцінюють ймовірність рідкісного класу і потребують подальшого коригування для коректної інтерпретації виходів.

Random Forest [9] є одним із класичних ансамблевих методів машинного навчання, що будує множину дерев рішень на різних bootstrap вибірках навчальних даних і агрегує їхні прогнози через усереднення або голосування. Кожне дерево навчається на випадково обраній підмножині прикладів та ознак, що забезпечує різноманітність компонентів ансамблю. Принциповою перевагою методу є висока стійкість до шуму у даних, мінімізація ризику перенавчання через усереднення множини дерев, можливість паралельного навчання та оцінка важливості ознак. Алгоритм базується на ідеях беггінгу, запропонованих Бреманою, і поєднує їх із технікою випадкового вибору ознак при побудові кожного вузла дерева. До обмежень належать значні вимоги до обчислювальних ресурсів і пам'яті при великих обсягах даних та глибоких деревах, а також обмежена інтерпретованість результатів порівняно з простими лінійними моделями.

Лінійна регресія [10] забезпечує моделювання лінійних залежностей між цільовою змінною та набором ознак. Незважаючи на простоту реалізації та інтерпретованість результатів, метод обмежений припущенням про лінійність зв'язків, що рідко виконується для задач аналізу структури графів і взаємозв'язків між вузлами кластера. У контексті задачі класифікації може використовуватись її варіант, а саме логістична регресія, проте без істотних модифікацій вона не дозволяє виявляти складні нелінійні взаємодії між ознаками, що є критичним недоліком для задачі аналізу матриць суміжності.

Метод k-найближчих сусідів [11] базується на пошуку у навчальній вибірці прикладів, що є найближчими до тестового прикладу за обраною метрикою, та формуванні прогнозу на основі їхніх міток. Метод є простим у реалізації та концептуально зрозумілим, проте погано масштабується на великі вибірки і страждає від так званого прокляття розмірності при високій кількості ознак. У задачі з 240 вимірним вектором ознак ефективність цей метод буде суттєво обмеженим через втрату значущості метричних відстаней у багатовимірному просторі.

Метод найвного байєсівського класифікатора [12] спирається на теорему Байєса і припущення про умовну незалежність ознак. Метод є обчислювально ефективним і потребує менше навчальних даних, проте припущення про незалежність ознак на практиці часто порушується, особливо у структурованих даних, таких як матриця суміжності графа. Для задачі аналізу топологій кластера, де зв'язки між вузлами є взаємозалежними, ця слабкість методу є принциповою.

Метод опорних векторів, він же й SVM, [12] будує гіперплощину, що оптимально розділяє приклади різних класів у багатовимірному просторі ознак. Метод є ефективним у просторах високої розмірності і дозволяє використання різних ядерних функцій для нелінійної класифікації. Основним обмеженням є висока обчислювальна вартість на великих навчальних вибірках та складність вибору відповідної ядерної функції. Для навчальних вибірок у понад мільйон прикладів, що використовуються у даній роботі, час навчання SVM стає неприйнятно великим.

Алгоритм адаптивного підсилення AdaBoost [13] є послідовним ансамблевим методом, у якому кожен наступний слабкий класифікатор навчається на прикладах, на яких помилялися попередні. Метод демонструє високу точність на задачах класифікації, проте є чутливим до шумів у даних і вимагає значних обчислювальних ресурсів при роботі з великими наборами. У контексті даної роботи AdaBoost не обраний через надмірну чутливість до випадкових варіацій у синтетичних даних.

Gradient Boosting [14] є узагальненням ідеї підсилення на основі градієнтного спуску у просторі функцій. Кожне наступне дерево ансамблю навчається мінімізувати градієнт функції втрат на залишкових помилках попередніх дерев. Метод дозволяє виявляти складні нелінійні залежності, є відносно стійким до шуму і має гнучкі можливості налаштування. Принциповою перевагою є те, що алгоритм природно обробляє ситуації з дисбалансом класів через відповідне налаштування ваг прикладів у функції втрат. До обмежень відноситься тривалий час навчання, чутливість до гіперпараметрів та ризик перенавчання без належної регуляризації.

LightGBM [15] є оптимізованою реалізацією градієнтного бустингу, що використовує гістограмний підхід до пошуку оптимальних розбиттів та листоорієнтовану стратегію зростання дерев. Метод вирізняється швидким навчанням та невеликим споживанням пам'яті, що робить його придатним для обробки великих обсягів даних. Однак LightGBM є більш чутливим до якості даних і потребує ретельного підбору гіперпараметрів. Хоча LightGBM не обраний як один з основних методів даної роботи, він має близькі характеристики до обраних алгоритмів і може розглядатись у майбутніх розширеннях.

CatBoost [16] є ще однією реалізацією градієнтного бустингу, розробленою з акцентом на коректну обробку категоріальних ознак та запобігання типовій для класичного бустингу проблемі цільового просочування. Метод використовує симетричні дерева, у яких всі вузли одного рівня застосовують одне й те саме розбиття, що діє як неявний регуляризатор і запобігає перенавчанню. Технологія впорядкованого бустингу, реалізована у CatBoost, математично гарантує відсутність просочування цільової змінної до ознак через використання випадкових перестановок навчальних даних і обчислення статистик категоріальних ознак лише на підмножині, що передує поточному прикладу. CatBoost демонструє високу стійкість до шуму, підтримує обчислення на графічних процесорах та надає засоби для контролю інтерпретованості моделі через візуалізацію важливості ознак.

Глибинне навчання [17] застосовує багат шарові штучні нейронні мережі для автоматичного виявлення складних нелінійних залежностей у даних. Метод демонструє високу точність у широкому колі задач, проте вимагає дуже великих обсягів навчальних даних, значних обчислювальних ресурсів та ретельного налаштування архітектури мережі. Для задач помірної розміру з обмеженими навчальними вибірками деревовидні ансамблі часто демонструють кращі результати, ніж глибокі нейронні мережі. У даній роботі глибинне навчання не обрано через невідповідність обчислювальним обмеженням та складність інтерпретації результатів.

Узагальнене порівняння розглянутих методів машинного навчання за критеріями придатності для задачі бінарної класифікації при дисбалансі класів наведено у табл. 1.1. У таблиці окремо зазначено три методи, обрані для подальшої роботи: Random Forest, Gradient Boosting та CatBoost.

Таблиця 1.1 – Порівняння методів машинного навчання за критеріями придатності

Метод	Переваги	Недоліки	Прийняття
Random Forest	Висока точність Стійкість до шуму Запобігання перенавчанню Паралельне навчання	Великі обсяги пам'яті при глибоких деревах Обмежена інтерпретованість	Обрано
Linear Regression	Простота Швидкість Прозорість	Лише лінійні залежності Чутливість до викидів	Не обрано
k-NN	Простота реалізації Адаптивність	Прокляття розмірності Повільність на великих наборах	Не обрано
Naive Bayes	Швидкість Ефективність на великих наборах	Припущення про незалежність ознак часто порушується	Не обрано
SVM	Ефективність у просторах високої розмірності Гнучкість налаштувань	Ризик перенавчання Повільність на великих вибірках	Не обрано
AdaBoost	Висока точність Послідовне навчання	Чутливість до шуму Складність масштабування	Не обрано
Gradient Boosting	Низька чутливість до шуму Виявлення складних залежностей	Тривалий час навчання Потреба у налаштуванні гіперпараметрів	Обрано

Продовження таблиці 1.1

LightGBM	Висока швидкість Низьке споживання пам'яті	Чутливість до якості даних Ризик перенавчання	Не обрано
CatBoost	Стійкість до шуму Вбудована робота з категоріями Підтримка GPU	Час навчання Складність інтерпретації	Обрано
Ridge Regression	Регуляризація проти мультиколінеарності	Лише лінійні залежності Неможливість відбору ознак	Не обрано
Lasso Regression	Автоматичний відбір ознак L1-регуляризація	Лише лінійні залежності Можлива упередженість оцінок	Не обрано
Deep Learning	Виявлення складних нелінійних залежностей	Потреба у дуже великих обсягах даних Обчислювальна вартість	Не обрано

Цей вибір обґрунтований їхньою здатністю ефективно працювати з великими навчальними вибірками при дисбалансі класів, відносною стійкістю до шуму та широкою документованістю реалізацій у Python екосистемі, що сприяє відтворюваності результатів. Усі три обрані методи є реалізаціями ансамблевого навчання на основі дерев рішень, що дозволяє у подальшому застосувати уніфікований підхід до калібрування і пошуку оптимального порогу класифікації для всіх моделей.

Окремим питанням є застосування ансамблевих метамоделей, що поєднують декілька базових моделей для отримання більш якісного агрегованого прогнозу. Ансамблеве навчання базується на ідеї, що при поєднанні декількох моделей із різними джерелами помилок можна отримати загальний прогноз, що є точнішим за прогнози окремих компонентів. Теоретичне обґрунтування цього підходу спирається на декомпозицію загальної помилки моделі на систематичне відхилення

(bias) та дисперсію. За рахунок усереднення дисперсія знижується пропорційно кількості компонентів, тоді як систематичне відхилення залежить від властивостей самих базових алгоритмів. Найпоширенішими стратегіями диверсифікації компонентів ансамблю є використання різних алгоритмів, навчання на різних підвибірках даних та налаштування з різними значеннями гіперпараметрів. У даній роботі реалізовано усі три стратегії ансамблювання, що детально описано у Розділі 3.

1.5 Постановка задачі

На основі проведеного аналітичного огляду формулюється завдання даної роботи. Метою є розробка та практична реалізація програмної системи бінарної класифікації конфігурацій мікросервісного кластера за станом ПРК на основі методів машинного навчання, а також експериментальне дослідження якості обраних моделей на синтетично згенерованих даних.

Формальна постановка задачі є такою. Нехай задано матрицю суміжності кластера A розміру $n \times n$, де $n \in [4, 15]$ — кількість вузлів у кластері, а елементи $A[i, j] \in \{0, 1\}$ визначають наявність двостороннього зв'язку між i -м та j -м вузлами. Кожному вузлу також зіставлено тип з обмеженого алфавіту $T = \{A, B, C\}$, що визначає виконуваний на ньому мікросервіс. Цільовою змінною $y \in \{0, 1\}$ є бінарна мітка, що приймає значення 1, якщо конфігурація кластера відповідає стану ПРК, і 0 у протилежному випадку. Мітка y формується детермінованим алгоритмом ізоляції зв'язних компонентів матриці суміжності з аналізом наявності типів у кожній компоненті.

Завданням є побудова статистичної моделі $f: X \rightarrow [0, 1]$, що приймає на вхід попередньо оброблене представлення матриці суміжності $x \in X$, яке отримане шляхом padding до фіксованої розмірності 240, і повертає оцінку ймовірності $P(Y=1|x)$ того, що відповідна конфігурація знаходиться у стані ПРК. Для забезпечення коректності числової інтерпретації виходу моделі застосовується

процедура ізотонічного калібрування на окремій вибірці з модельним розподілом класів. Після калібрування виходи моделі можуть інтерпретуватись як справжні ймовірності і використовуватись для ранжування конфігурацій за рівнем ризику.

Цільові вимоги до результатів роботи наступні. По-перше, реалізована модель має забезпечувати високі значення показника ROC-AUC та PR-AUC на тестовій вибірці, що свідчитиме про здатність моделі відрізняти конфігурації різних класів за виходом. По-друге, числові виходи моделі після калібрування мають відповідати реальній частоті позитивного класу у тестовій вибірці, що оцінюється показником Expected Calibration Error. По-третє, система має забезпечувати повну відтворюваність експериментів через фіксацію генераторів псевдовипадкових чисел та публічне розміщення вихідного коду у відкритому репозиторії.

Окрім цих кількісних критеріїв, робота має задовольняти ряд якісних вимог. Реалізація повинна бути модульною, що дозволяє замінити або розширити окремі компоненти без переробки всієї системи. Архітектура має дотримуватись принципів чистого коду і документованості, що полегшує супровід та подальший розвиток. Веб-інтерфейс системи має бути інтуїтивно зрозумілим для користувачів без глибокого знання деталей внутрішньої реалізації моделей машинного навчання.

Слід зазначити декілька важливих особливостей постановки задачі, що визначають характер дослідження. Цільова мітка у є детермінованою функцією від матриці суміжності, що обчислюється алгоритмом пошуку у глибину за час $O(V+E)$. У такому контексті розроблена модель машинного навчання є статистичним наближенням алгоритмічної функції на множині синтетично згенерованих топологій. Доцільність такого наближення полягає у наступних аспектах. Воно дозволяє продемонструвати повний цикл побудови, калібрування та порівняльної оцінки моделей бінарної класифікації при значному дисбалансі класів. Дослідження створює методологічну базу для майбутніх розширень системи на сценарії з обмеженою спостережуваністю топології, де повна матриця суміжності недоступна. Воно забезпечує інструмент для архітекторів розподілених

систем, що дозволяє кількісно оцінювати конфігурації на етапі їхнього проектування і порівнювати альтернативні топології за схильністю до ПРК.

Зазначені обмеження дослідження є такими. Усі експерименти проводяться на синтетично згенерованих даних з використанням моделі випадкових графів Ердьош-Реньї, що накладає обмеження на безпосередній перенос результатів на реальні промислові кластери, у яких зв'язки між вузлами не є незалежними. Діапазон розмірів кластера обмежено значеннями від 4 до 15 вузлів через обмеження пам'яті при генерації та обробці навчальних вибірок. Топології є статичними і не передбачають еволюції у часі, тому модель не виявляє передвісників ПРК у динамічному сенсі, а лише оцінює поточну конфігурацію за вмонтованою функцією.

Конкретні завдання, що мають бути розв'язані для досягнення поставленої мети, є такими. Сформулювати методiku генерації синтетичних навчальних, калібрувальних та тестових вибірок з модельним розподілом класів. Реалізувати три базові моделі машинного навчання, а саме Random Forest, Gradient Boosting, CatBoost із належним урахуванням дисбалансу класів. Реалізувати три ансамблеві моделі з різними стратегіями диверсифікації компонентів, що дозволяє порівняти ефективність окремих стратегій ансамблювання. Розробити процедуру ізотонічного калібрування ймовірностей на окремій вибірці. Реалізувати декілька методів пошуку оптимального порогу класифікації, що відповідають різним сценаріям використання системи. Провести експериментальне дослідження реалізованих моделей на тестовій вибірці з аналізом метрик якості до і після калібрування. Реалізувати веб-інтерфейс для інтерактивної демонстрації роботи системи з графічним редактором топології кластера.

Очікувані результати роботи мають дослідницький і практичний характер. З дослідницької точки зору очікується отримання кількісних оцінок придатності обраних методів машинного навчання для апроксимації структурних властивостей графів кластерів та емпіричної перевірки ефективності калібрування і ансамблювання. Окремим очікуваним результатом є виявлення сильних і слабких

сторін кожного з трьох обраних методів та трьох ансамблевих стратегій у контексті даної задачі. З практичної точки зору очікується розробка повного програмного комплексу із публічним вихідним кодом, який може бути використаний для аналізу конфігурацій кластерів архітекторами розподілених систем та для подальшого розвитку дослідницьким співтовариством.

Висновки до розділу 1

У першому розділі проведено аналітичний огляд предметної області, що дозволив сформувати наукове підґрунтя для подальшої роботи. Розглянуто особливості розподілених інформаційних систем та мікросервісної архітектури, проаналізовано фундаментальні обмеження таких систем, описано різні типи мережових розривів та сценарії виникнення ПРК. Сформульовано практичну потребу в інструментах кількісної оцінки структурної крихкості топологій кластерів на етапі їхнього проектування.

Проведено детальний огляд відомих алгоритмічних підходів до запобігання ПРК. Показано, що жоден з цих підходів не дозволяє оцінювати конфігурацію кластера до її розгортання, що формує практичну нішу для застосування методів машинного навчання як інструментарію аналізу.

У результаті порівняльного огляду дванадцяти методів машинного навчання обґрунтовано вибір трьох найбільш придатних алгоритмів для розв'язання поставленої задачі, а саме Random Forest, Gradient Boosting та CatBoost. Сформульовано формальну постановку задачі бінарної класифікації конфігурацій кластера за станом ПРК, визначено вхідні дані, цільову змінну, вимоги до якості результатів та основні обмеження дослідження.

2 МЕТОДОЛОГІЯ ДОСЛІДЖЕННЯ

2.1 Загальна архітектура системи

Система, що розробляється у даній роботі, є програмним комплексом для бінарної класифікації конфігурацій мікросервісних кластерів за станом ПРК. Архітектура системи проектується з урахуванням двох сценаріїв використання. Перший сценарій передбачає інтерактивну роботу архітектора кластерних систем з графічним веб-інтерфейсом для побудови довільних топологій та отримання оцінок ймовірності ПРК у реальному часі. Другий сценарій передбачає дослідницьку роботу з пайплайном навчання моделей через інтерфейс командного рядка, що включає генерацію навчальних даних, тренування базових та ансамблевих моделей, калібрування ймовірностей та пошук оптимальних порогів класифікації.

Двоїстість сценаріїв використання визначає основні архітектурні рішення системи. Для забезпечення інтерактивної взаємодії з користувачем реалізовано веб-застосунок на основі фреймворку Django, що надає графічний редактор топологій та доступ до навчених моделей через інтерфейс. Для забезпечення відтворюваності дослідницьких експериментів реалізовано модульний пайплайн навчання, кожен етап якого може запускатись незалежно через відповідні скрипти. Зв'язок між цими частинами системи реалізовано через спільне сховище серіалізованих моделей у форматі pkl і JSON файлів з параметрами оптимальних порогів.

Сценарій використання застосунку через інтерфейс зображено на рис. 2.1. На діаграмі виділено можливості по дослідженню та використанню системи.

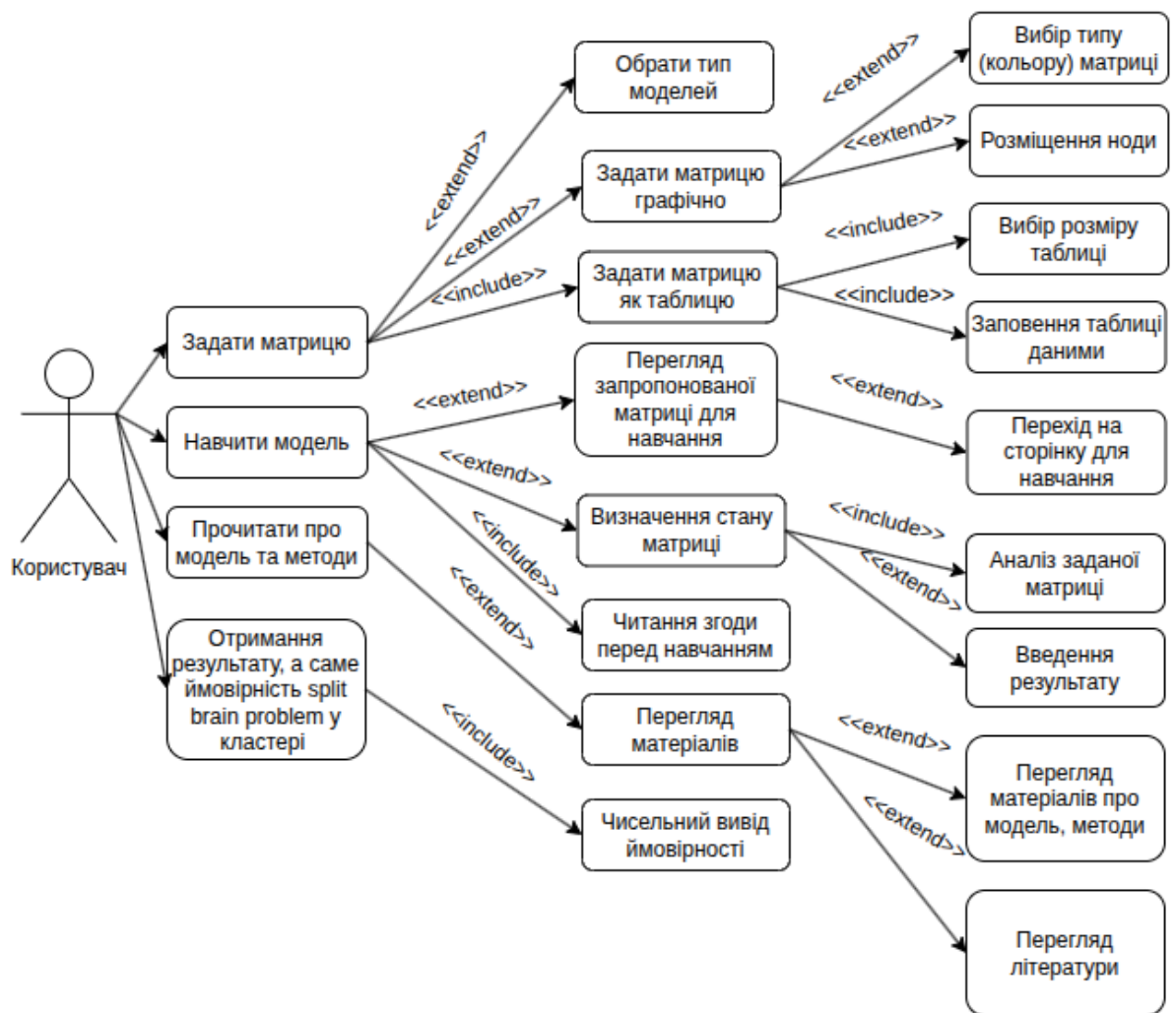


Рисунок 2.1 – Діаграма варіантів використання системи

Внутрішня структура системи представлена діаграмою потоків обробки запиту на рис. 2.2. Діаграма відображає основні функціональні модулі системи та зв'язки між ними. Презентаційний шар включає веб-інтерфейс на основі HTML5 Canvas та JavaScript, що забезпечує інтерактивну роботу з топологією. Веб-сервер реалізований на фреймворку Django і виконує маршрутизацію HTTP запитів, рендеринг шаблонів та координацію викликів до модулів обробки даних і інференсу. Модуль обробки даних реалізує функції перетворення матриці суміжності у вектор ознак, а також функції алгоритмічного визначення стану кластера.

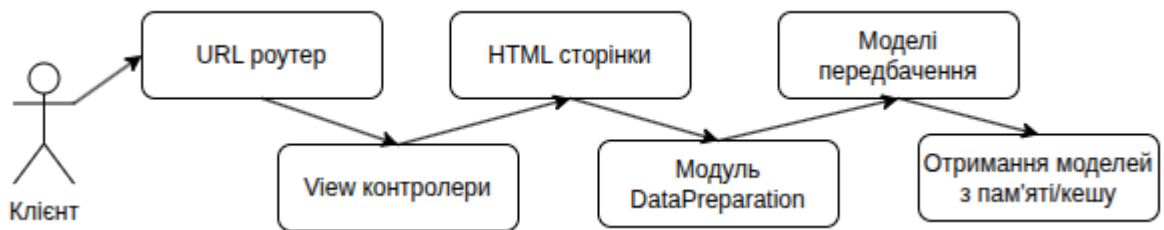


Рисунок 2.2 – Діаграма потоків обробки запиту

Модуль інференсу відповідає за завантаження серіалізованих моделей зі сховища, їхнє кешування у пам'яті процесу та виклик відповідних методів прогнозування. Кешування у пам'яті є важливою оптимізацією, оскільки завантаження великих pickle файлів, до прикладу Random Forest, що може досягати декількох гігабайт, є повільною операцією, а виклик прогнозу на вже завантаженій моделі займає лічені мілісекунди. Базові моделі представлені трьома незалежно навченими класифікаторами CatBoost, Gradient Boosting та Random Forest. Ансамблеві моделі є метамоделями, що поєднують декілька базових моделей через зважене усереднення каліброваних прогнозів.

Пайплайн навчання є відокремленим модулем, що використовує генератор даних для створення синтетичних навчальних вибірок та послідовно навчає всі моделі системи. Він не залучається при обробці запитів через веб інтерфейс, проте формує сховище моделей, що використовується іншими модулями. Сховище моделей є фізичним каталогом файлової системи, у якому зберігаються серіалізовані моделі у форматі pickle та параметри оптимальних порогів класифікації у форматі JSON. Така організація забезпечує чітке розмежування обов'язків між компонентами і дозволяє замінювати або оновлювати окремі модулі без впливу на решту системи.

2.2 Формальна постановка задачі

У даному підрозділі надається формальне математичне формулювання задачі, що розв'язується у роботі. Нехай задано множину можливих топологій мікросервісних кластерів, кожна з яких представлена графом $G = (V, E)$, де V — множина вузлів графа (нод кластера), а $E \subseteq V * V$ — множина ребер. Завжди передбачається двосторонніх зв'язків між нодами, як повноцінний. Розмір графа $n = |V|$ є змінним і знаходиться у діапазоні $n \in [4, 15]$. Кожному вузлу $v \in V$ зіставлено тип з алфавіту $T = \{A, B, C, \dots, Z\}$, що визначає мікросервіс, який виконується на даному вузлі.

Топологія кожного графа представляється матрицею суміжності A розміру $n \times n$, де $A[i, j] = 1$, якщо між вузлами i та j існує двосторонній зв'язок, і $A[i, j] = 0$ у протилежному випадку. Матриця є симетричною ($A[i, j] = A[j, i]$) і має нульову діагональ ($A[i, i] = 0$), що відображає неможливість самозв'язку вузла. Інформація про типи вузлів зберігається окремо у вигляді вектора типів $\tau = (\tau_1, \tau_2, \dots, \tau_n)$, де $\tau_i \in T$.

Цільова функція маркування є $u: (A, \tau) \rightarrow \{0, 1\}$ визначається як індикатор стану ПРК. Функція повертає значення 1, якщо кластер з даною топологією знаходиться у стані розщеплення, і 0 у протилежному випадку. Формально функція реалізується алгоритмом ізоляції зв'язних компонент матриці суміжності з подальшим аналізом наявності всіх типів вузлів у кожній компоненті. Кластер вважається у стані ПРК, якщо принаймні дві з його зв'язних компонент містять повний набір типів і таким чином можуть незалежно функціонувати як повноцінні підкластери.

Оскільки вхідні дані мають змінну розмірність, а більшість алгоритмів машинного навчання потребують вхідних векторів фіксованої розмірності, необхідним є перетворення вхідних даних у уніфікований формат. Це перетворення позначається як функція $\phi: (A, \tau) \rightarrow \mathbb{R}^d$, де $d = 240$ — розмірність вектора ознак. Функція реалізується через техніку padding, детально описану у підрозділі 2.4.

Підсумкова постановка задачі полягає у побудові статистичної моделі $\hat{f}: \mathbb{R}^d \rightarrow [0, 1]$, що для будь-якого вектора ознак $x = \varphi(A, \tau)$ повертає оцінку ймовірності $P(Y=1|x)$ того, що відповідна конфігурація знаходиться у стані ПРК.

Окрім самої функції $\hat{f}$, у системі реалізується процедура калібрування $g: [0, 1] \rightarrow [0, 1]$, що коригує сирі виходи моделі для забезпечення відповідності між прогнозованими ймовірностями та реальними частотами позитивного класу. Після калібрування модель приймає вигляд $\hat{f}_{\text{cal}}(x) = g(\hat{f}(x))$. Для прийняття бінарних рішень також визначається поріг класифікації $\tau \in [0, 1]$, такий що бінарний прогноз $\hat{y}$ обчислюється як $\hat{y} = 1$, якщо $\hat{f}_{\text{cal}}(x) \geq \tau$, та $\hat{y} = 0$ у протилежному випадку. Вибір порогу залежить від сценарію використання і виконується одним з декількох методів, що детально описуються у Розділі 3.

2.3 Генерація синтетичних даних

Оскільки реальні дані про конфігурації мікросервісних кластерів у стані ПРК у відкритому доступі відсутні, для проведення експериментів використовуються синтетично згенеровані дані. Вони дозволяють створювати вибірки довільного обсягу, контролювати розподіл класів, забезпечувати відтворюваність експериментів через фіксацію генератора псевдовипадкових чисел. Водночас синтетичні дані мають і принципове обмеження, бо вони відображають властивості саме обраної моделі генерації, що не обов'язково збігаються з властивостями реальних промислових кластерів.

Для генерації топологій застосовується модель випадкових графів Ердьош-Реньї, що є однією з найпростіших і найбільш вивчених моделей у теорії випадкових графів. Згідно з цією моделлю, для заданої кількості вузлів n кожна можлива пара вузлів незалежно отримує ребро з фіксованою ймовірністю $p_{\text{link}} \in [0, 1]$. Параметр p_{link} є основним керованим параметром моделі і визначає очікувану щільність зв'язків у графі, при $p_{\text{link}} = 0$ граф взагалі не має ребер, при $p_{\text{link}} = 1$ граф є повним.

Алгоритм генерації одного прикладу складається з декількох послідовних кроків. По-перше, випадково обирається розмір кластера n з рівномірного розподілу на діапазоні $[4, 15]$. По-друге, для кожного з n вузлів випадково обирається тип з алфавіту $T = \{A, B, C\}$ з рівними ймовірностями. По-третє, для кожної пари вузлів незалежно генерується наявність ребра з ймовірністю $p_{\text{link}} = 0.7$. По-четверте, отримана матриця суміжності проходить перевірку алгоритмом `isClusterDead` на наявність мертвого стану, а саме такого, коли є відсутність хоча б одного типу мікросервісу в усіх зв'язних компонентах, і у разі позитивного результату приклад відкидається і генерується наново. По-п'яте, обчислюється цільова мітка у через виклик функції `isSplitBrain`.

Функція `isSplitBrain` реалізує детермінований алгоритм визначення стану ПРК. Алгоритм використовує пошук у глибину для виявлення всіх зв'язних компонент графа, поданого матрицею суміжності. Для кожної знайденої компоненти перевіряється, чи містить вона повний набір типів $\{A, B, C\}$. У такому випадку компонента може функціонувати як самостійний кластер. Якщо принаймні дві такі повноцінні компоненти існують одночасно, кластер вважається у стані ПРК і повертається з міткою 1. Якщо існує лише одна повноцінна компонента або жодної, кластер не у стані ПРК, що відповідає мітці 0.

Аналогічно функція `isClusterDead` визначає, чи є кластер взагалі функціональним. Кластер вважається мертвим, якщо у жодній з його зв'язних компонент немає повного набору типів $\{A, B, C\}$. У такому стані кластер не може обслуговувати транзакції, що вимагають взаємодії всіх типів мікросервісів. Мертві кластери виключаються з навчальної вибірки, оскільки вони відповідають патологічному стану, не пов'язаному безпосередньо з проблемою ПРК.

Експериментально визначено, що при заданих параметрах генератора модельна частка прикладів зі станом ПРК у випадково згенерованих кластерах становить приблизно 9%. Слід підкреслити, що ця частка є властивістю саме обраної моделі генерації, а не реальних промислових кластерів. У реальних системах зв'язки між нодами не є незалежними, а топології формуються з

урахуванням таких факторів як географічне розташування дата-центрів, мережева інфраструктура та політики безпеки. Тому модельну частку 9% слід інтерпретувати як характеристику синтетичних даних, що використовуються для дослідження, а не як універсальний природний рівень ПРК.

Модельна частка 9% означає значний дисбаланс класів. Приблизно 91% прикладів належать до негативного класу, і лише 9% до позитивного. Такий дисбаланс ускладнює навчання моделей машинного навчання, оскільки оптимізація стандартної функції втрат призводить до зміщення прогнозів у бік мажоритарного класу. Для подолання цього обмеження застосовується штучне збагачення тренувальної вибірки прикладами позитивного класу до цільової частки 35%. Алгоритм збагачення є таким. Окремо генеруються природні приклади, з модельним розподілом у 9% ПРК, та форсовані приклади ПРК, яка гарантовано мають позитивну мітку через спеціальний генератор. Кількості обох категорій підбираються таким чином, щоб результуюча тренувальна вибірка мала цільову частку ПРК.

Принципово важливим аспектом методології є те, що штучне збагачення застосовується тільки до тренувальної вибірки, тоді як калібрувальна і тестова вибірки зберігають модельний розподіл у приблизно 9% ПРК. Це дозволяє моделі навчатись на достатній кількості прикладів позитивного класу, а наступне калібрування коригує її виходи відповідно до природного співвідношення класів. Без такого розділення модель систематично переоцінювала б ймовірність ПРК на нових даних, що робило б її практичну користь сумнівною.

2.4 Перетворення даних та формування ознак

Як зазначено у підрозділі 2.2, вхідні дані мають змінну розмірність, від 4 на 4 до 15 на 15 для матриці суміжності, а обрані алгоритми машинного навчання потребують вхідних векторів фіксованої розмірності. Це робить необхідним проміжний етап перетворення вхідних даних до уніфікованого формату. У даній

роботі застосовується техніка padding, що полягає у доповненні матриці спеціальними маркерами до фіксованого розміру.

Обрана стратегія padding передбачає перетворення матриці суміжності A розміру $n \times n$ у матрицю фіксованого розміру 16×15 . Перший рядок цієї фіксованої матриці містить кодування типів вузлів. Цілі числа, що позначають тип кожної з 15 можливих позицій. Тип A кодується як 2, тип B — як 3, тип C — як 4, тощо. Позиції, що не використовуються через менший розмір реального кластера, заповнюються значенням -1. Наступні n рядків містять відповідні рядки оригінальної матриці суміжності, при цьому стовпці, що виходять за межі реального розміру кластера, заповнюються значенням -1. Решта рядків повністю заповнюється значенням -1.

Загальна розмірність отриманого вектора ознак становить 240. Це значення обрано з міркувань врахування максимально можливого розміру кластера з резервом одного додаткового рядка для кодування типів. Використання значення -1 як маркера відсутності даних дозволяє моделям машинного навчання відрізнити реальні нульові зв'язки від штучно доданих заповнювачів. Хоча алгоритми деревовидних ансамблів безпосередньо не підтримують механізм маскування відсутніх значень, вони здатні навчитись розрізняти ці два випадки за рахунок різниці у числових значеннях.

Перетворення матриці у вектор виконується шляхом конкатенації її рядків. Результируючий вектор передається на вхід моделей машинного навчання як одновимірний масив із 240 числових ознак. Така стратегія є простою у реалізації та не потребує спеціальної підтримки з боку алгоритмів навчання, що робить її сумісною з усіма обраними методами.

Альтернативними підходами до уніфікації представлення графів змінної розмірності можуть бути спектральні методи, зокрема, перетворення Лапласа або графове перетворення Фур'є, що дозволяють отримати вектор ознак сталої довжини незалежно від кількості вузлів. Проте такі методи мають й свої обмеження. Спектральні коефіцієнти можуть мати комплексні значення, що ускладнює подальше використання у моделях, орієнтованих на дійсні числа. Крім того,

спектральні методи добре передають глобальні властивості графа, але можуть втрачати локальну специфіку, що є критично важливою для виявлення ПРК. Тому в даній роботі обрано простіший підхід padding, що зберігає повну інформацію про матрицю суміжності.

2.5 Розбиття даних на вибірки

Коректне розбиття даних на незалежні вибірки є критичним аспектом методології машинного навчання, що безпосередньо впливає на достовірність отриманих результатів. У даній роботі використовується чотири окремі вибірки, кожна з яких має чітко визначене призначення і не перетинається з іншими. Структура та призначення кожної вибірки наведено у табл. 2.1.

Таблиця 2.1 – Розбиття даних на незалежні вибірки

Вибірка	Розмір	Розподіл класів	Призначення
Тренувальна	1100000	збагачена (35% ПРК)	навчання моделей
Валідаційна для early stop	20000	модельний (9% ПРК)	контроль перенавчання
Калібрувальна	8000	модельний (9% ПРК)	ізотонічне калібрування
Тестова	20000	модельний (9% ПРК)	фінальна оцінка моделей

Тренувальна вибірка є найбільшою за обсягом і використовується безпосередньо для підбору параметрів моделей. Її розподіл класів збагачено штучно згенерованими прикладами ПРК до цільової частки 35%, що дозволяє моделям краще навчитись виявленню позитивного класу. Обсяг тренувальної вибірки становить 1100000 прикладів для моделей CatBoost і Gradient Boosting та

500000 прикладів для Random Forest. Зменшений обсяг для Random Forest обумовлений обмеженнями за обсягом пам'яті при серіалізації дерев.

Валідаційна вибірка для ранньої зупинки є невеликою за обсягом і використовується алгоритмами градієнтного бустингу для моніторингу якості моделі у процесі навчання. У реалізації CatBoost ця вибірка передається через параметр `eval_set` і використовується для визначення моменту зупинки навчання, коли якість на валідації перестає поліпшуватися протягом заданої кількості ітерацій. На відміну від тренувальної вибірки, валідаційна вибірка має модельний розподіл класів, що відповідає природному розподілу для генератора. Це дозволяє моделі коригувати свою поведінку відповідно до того розподілу, який вона зустрічатиме на тестовій вибірці.

Калібрувальна вибірка є окремою і використовується виключно для процедури ізотонічного калібрування, що детально описується у Розділі 3. Її обсяг становить 8000 прикладів, що є достатнім для стабільного підбору монотонної функції перетворення без істотного ризику перенавчання калібруатора. Розподіл класів у калібрувальній вибірці також відповідає модельному, що є принциповою вимогою. Калібрування має приводити виходи моделі у відповідність до тих самих частот, з якими модель зустрічатиметься на тестовій вибірці.

Тестова вибірка використовується для фінальної оцінки якості моделей і не задіюється на жодному з попередніх етапів навчання. Її обсяг становить 20000 прикладів, що забезпечує достатньо репрезентативну оцінку метрик якості. Розподіл класів у тестовій вибірці є модельним, що відповідає очікуваному розподілу при роботі системи з реальними топологіями. Звичайно це з урахуванням того, що реальний розподіл є властивістю моделі генератора, як обговорено у підрозділі 2.3.

Принципово важливо, що всі чотири вибірки генеруються незалежно одна від одної з фіксованими, але різними значеннями `random seed`. Це гарантує, що між вибірками немає жодних повторюваних прикладів, що могло б призвести до оптимістично зміщених оцінок якості. Незалежність калібрувальної та тестової

вибірок є особливо критичною. Якщо калібратор підлаштовується під ту саму вибірку, на якій згодом обчислюються метрики, отримані оцінки будуть оптимістичними і не відображатимуть реальну якість системи на нових даних.

2.6 Метрики оцінки якості моделей

Для оцінки якості реалізованих моделей машинного навчання у даній роботі використовується комплексний набір метрик, що покривають різні аспекти якості бінарної класифікації. Вибір саме такого набору обумовлений специфікою задачі, а саме значним дисбалансом класів, необхідністю калібрування ймовірностей та потребою у виборі оптимального порогу класифікації. Узагальнений перелік використовуваних метрик наведено у табл. 2.2.

Таблиця 2.2 – Метрики оцінки якості моделей

Метрика	Що вимірює	Інтерпретація
ROC-AUC	Розрізнявальну здатність моделі за всіма порогоми	1.0 — ідеально, 0.5 — випадково
PR-AUC	Якість прогнозів позитивного класу при дисбалансі	Базовий рівень — частка позитивного класу
Precision	Частка істинно позитивних серед усіх прогнозованих позитивними	1.0 — немає хибних тривог
Recall	Частка виявлених позитивних серед усіх реальних позитивних	1.0 — жодного пропуску ПРК
F1-score	Гармонічне середнє Precision і Recall	Збалансована оцінка точності
FPR	Частка хибно позитивних серед усіх реальних негативних	0.0 — немає хибних тривог

Продовження таблиці 2.2

ECE	Середнє абсолютне відхилення між прогнозованою та фактичною ймовірністю	0.0 — ідеальне калібрування
Brier score	Середньоквадратичне відхилення прогнозу від мітки	0.0 — ідеальний прогноз

Метрики ROC-AUC та PR-AUC оцінюють якість моделі за всіма можливими порогами класифікації і є незалежними від конкретного значення порогу. ROC-крива відображає залежність частки істинно позитивних прогнозів від частки хибно позитивних прогнозів при варіюванні порогу. Площа під ROC-кривою є інтегральною мірою розрізнявальної здатності моделі. Значення 1.0 відповідає ідеальній моделі, що повністю розділяє класи, при цьому значення 0.5 відповідає випадковому класифікатору.

PR-крива відображає залежність точності від повноти при варіюванні порогу. Площа під PR-кривою є особливо інформативною при значному дисбалансі класів, оскільки на відміну від ROC-AUC вона не розмивається великою кількістю істинно негативних прогнозів. У задачі з модельною часткою позитивного класу 9% базовим рівнем PR-AUC є саме 0.09. Значення PR-AUC, суттєво вище за цей базовий рівень, свідчить про корисну розрізнявальну здатність моделі.

Метрики Precision, Recall, F1-score та FPR оцінюються для конкретного порогу класифікації. Precision визначається як частка істинно позитивних прогнозів серед усіх прогнозів, позначених моделлю як позитивні. Recall визначається як частка істинно позитивних прогнозів серед усіх реально позитивних прикладів. F1-score є гармонічним середнім Precision і Recall, що приймає високі значення лише тоді, коли обидві компоненти є одночасно високими. FPR визначається як частка хибно позитивних прогнозів серед усіх реально негативних прикладів. Усі ці метрики приймають значення з діапазону [0, 1].

Метрики ECE та Brier score оцінюють якість калібрування ймовірностей. ECE обчислюється як зважене середнє абсолютне відхилення між прогнозованою

ймовірністю та реальною частотою позитивного класу по фіксованій кількості бінів значень ймовірностей. Для добре відкаліброваної моделі ECE має бути близьким до нуля, що означає, що серед усіх прикладів з прогнозованою ймовірністю p приблизно частка p дійсно належить до позитивного класу. Brier score є середньоквадратичним відхиленням прогнозованої ймовірності від реальної мітки і одночасно враховує як точність прогнозу, так і якість калібрування. Низькі значення Brier score свідчать про добре відкалібровану модель з високою точністю.

Усі перелічені метрики обчислюються на тестовій вибірці з модельним розподілом класів. Для метрик, що залежать від порогу класифікації, значення обчислюються для кожного з трьох методів вибору порогу: метод Юдена, метод MaxF1 та метод High-Recall. Це дозволяє порівняти різні стратегії компромісу між хибно позитивними та хибно негативними помилками в умовах конкретної задачі.

Узагальнюючи, обраний набір метрик дозволяє всебічно оцінити якість реалізованих моделей. ROC-AUC і PR-AUC характеризують загальну розрізнявальну здатність незалежно від порогу. Precision, Recall, F1-score та FPR характеризують якість прогнозів при конкретних порогах. ECE та Brier score характеризують якість калібрування ймовірностей. Така комплексна оцінка є необхідною для обґрунтованого порівняння реалізованих моделей між собою та для вибору найбільш придатної архітектури.

Висновки до розділу 2

У другому розділі представлено методологію дослідження. Розроблено загальну архітектуру системи з двома основними сценаріями використання системи. Архітектура відображена у вигляді діаграми варіантів використання та компонентної діаграми, що демонструє розподіл функціональності між п'ятьма основними модулями системи.

Сформульовано формальну постановку задачі бінарної класифікації. Дано матрицю суміжності кластера змінного розміру, потрібно побудувати функцію

оцінки ймовірності перебування кластера у стані ПРК. Описано методику генерації синтетичних даних за моделлю випадкових графів Ердьош-Реньї з параметром $p_{\text{link}} = 0.7$, при якому модельна частка позитивних прикладів становить приблизно 9%. Запропоновано padding стратегію перетворення матриць змінного розміру у вектор фіксованої розмірності 240, що забезпечує сумісність з усіма обраними алгоритмами машинного навчання.

Особливу увагу приділено коректному розбиттю даних на чотири незалежні вибірки. Зведено перелік метрик якості моделей з обґрунтуванням їхньої придатності для задач з дисбалансом класів.

3 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ

3.1 Технологічний стек

Реалізацію системи виконано на мові програмування Python 3.10 [18], яка є одним з найкращих виборів для прикладного машинного навчання завдяки розвиненій екосистемі бібліотек, читабельному синтаксису та широкій спільноті розробників. Вибір саме третьої версії Python обумовлений наявністю підтримки типізації та сучасних мовних конструкцій, що покращують зрозумілість коду. Усі залежності проєкту явно фіксовані у файлі requirements.txt з точними версіями, що забезпечує відтворюваність експериментів на інших обчислювальних платформах.

Реалізація моделі CatBoost використовує однойменну бібліотеку [16]. Бібліотека написана мовою C++ для забезпечення високої продуктивності та має Python інтерфейс, що дозволяє інтегрувати її у пайплайн обробки даних без втрати швидкості. CatBoost підтримує паралельне навчання на CPU та використання графічних процесорів через CUDA, що значно прискорює час навчання на великих вибірках. Обрана для роботи версія 1.2 містить усі ключові функції, необхідні для побудови класифікаторів: автоматичну обробку категоріальних ознак, ізотонічне калібрування, ранню зупинку навчання та серіалізацію моделей у бінарний формат.

Моделі Gradient Boosting та Random Forest реалізовані через бібліотеку scikit-learn [19], яка є добре документованою бібліотекою машинного навчання. Бібліотека надає уніфікований інтерфейс для всіх класифікаторів, що спрощує підстановку різних моделей у єдиний пайплайн без зміни оточуючого коду. Окрім самих класифікаторів, у роботі використовуються допоміжні модулі scikit-learn. CalibratedClassifierCV для ізотонічного калібрування, метрики ROC, PR та інші стандартні інструменти оцінки якості моделей.

Допоміжна бібліотека NumPy [20] використовується для всіх операцій з матрицями і векторами числових даних. NumPy надає ефективну реалізацію багатовимірних масивів через C-бекенд, що дозволяє виконувати векторизовані

обчислення на порядки швидше за чистий Python-цикл. Перетворення матриці суміжності у вектор ознак фіксованої довжини, обчислення метрик якості моделей та підготовка вибірок здійснюються саме через NumPy.

Веб-інтерфейс системи реалізовано на основі фреймворку Django [21], який є зрілим і широко використовуваним рішенням для розробки веб-застосунків мовою Python. Django надає інтегровану архітектуру MVT, вбудовану підтримку маршрутизації http запитів, систему шаблонізації та інструменти для обробки форм. Безпосередня інтеграція моделей машинного навчання у Django контролери виконується без необхідності у міжмовній взаємодії. В свою чергу, pickle файли завантажуються через стандартний модуль pickle і використовуються як звичайні об'єкти.

Серіалізація навчених моделей виконується через стандартний модуль pickle Python. Pickle є простим у використанні форматом і дозволяє зберігати будь-які Python об'єкти, включаючи складні моделі CatBoost, Random Forest та власні класи ансамблів. Альтернативами могли бути формати JSON або спеціалізовані бінарні формати. Однак pickle обрано через універсальність. Він однаково добре працює з усіма типами моделей у проєкті, що спрощує архітектуру модуля інференсу. Загальну структуру пайплайну навчання моделей наведено на рис. 3.1.

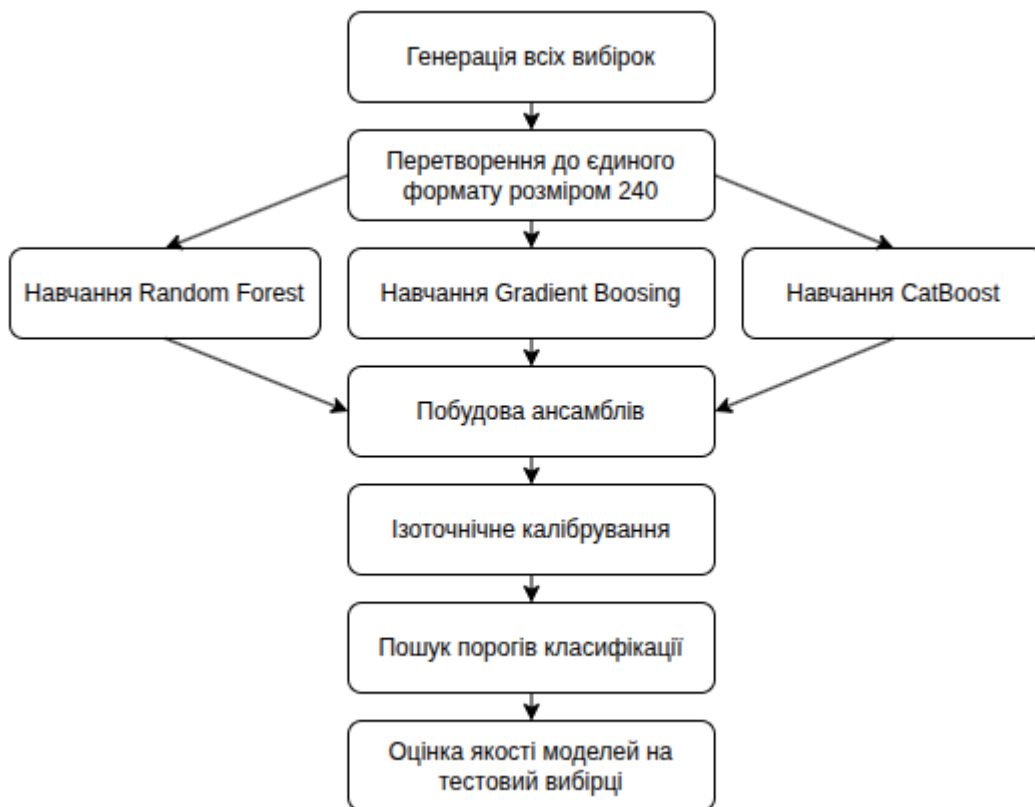


Рисунок 3.1 – Пайплайн навчання моделей

Пайплайн організовано як послідовність етапів, кожен з яких приймає результат попереднього і передає свій вихід наступному. Така архітектура полегшує діагностику проблем. При виникненні відхилень у фінальних метриках якості можна послідовно перевіряти кожен етап і визначати, на якому з них з'являється помилка.

3.2 Реалізація базових моделей

Базові моделі є фундаментом архітектури системи. Вони безпосередньо навчаються на вибірках синтетичних кластерів і використовуються як автономні класифікатори, а також як компоненти ансамблевих архітектур. У роботі реалізовано три базові моделі: CatBoost, Gradient Boosting та Random Forest. Усі три моделі мають уніфікований інтерфейс через метод `predict_proba`, що повертає двовимірний масив ймовірностей для негативного і позитивного класів. Підсумкові

гіперпараметри усіх трьох моделей наведено у табл. 3.1, а нижче надається детальний опис реалізації кожної.

Таблиця 3.1 – Конфігурація базових моделей

Параметр	CatBoost	Gradient Boosting	Random Forest
Кількість дерев	до 3000	до 500	600
Глибина дерев	7	9	15
Темп навчання (lr)	0.05	0.05	-
Субвибірка прикладів	-	0.8	bootstrap
Субвибірка ознак	-	0.8	15
Мінімум у листку	min_data_in_leaf=10	-	min_samples_leaf=4
Регуляризація	l2_leaf_reg=5.0	subsample	min_samples_leaf
Балансування класів	scale_pos_weight = 10.1	sample_weight = 10.1	balanced_subsample
Обсяг навчання	1100000	1100000	500000
Функція втрат	Logloss	deviance	Gini
Паралелізація	thread_count=-1	однопотокова	n_jobs=-1
Рання зупинка	early_stop=100	n_iter_no_change=20	-

3.2.1 Реалізація CatBoost

Модель CatBoost навчається через клас CatBoostClassifier з ключовими параметрами iterations = 3000, depth = 7, learning_rate = 0.05, loss_function = "Logloss", eval_metric = "AUC". Обмеження глибини на рівні 7 рівнів є типовим для

CatBoost і забезпечує достатню виразність моделі без надмірного перенавчання за рахунок неявної регуляризації через симетричність дерев.

Балансування класів реалізується через параметр `scale_pos_weight`, що дорівнює $\text{NATURAL_SPW} = (1 - 0.09) / 0.09 \approx 10.1$, обчисленому від модельної частки ПРК у генераторі. Це значення є спільним для всіх компонентів ансамблів типу `E1_Mixed` та `E3_CB_Hyper`, тоді як для `E2_CB_Bias` значення обчислюється індивідуально для кожного компонента залежно від його тренувальної вибірки. Винесення `NATURAL_SPW` у явну константу, що використовується усіма моделями, є принциповим архітектурним рішенням, що забезпечує консистентність балансування класів між базовими моделями і ансамблями.

Рання зупинка навчання активується при відсутності покращення метрики AUC на валідаційній вибірці протягом 100 ітерацій. Після спрацювання ранньої зупинки модель автоматично відновлюється до стану ітерації з найкращим значенням AUC завдяки параметру `use_best_model = True`. Цей механізм є ключовим для запобігання перенавчанню. Хоча теоретично можливі 3000 ітерацій, фактично навчена модель зупиняється значно раніше, у точці найкращого балансу між точністю на навчальній вибірці та здатністю до узагальнення.

Регуляризація CatBoost здійснюється через параметр `l2_leaf_reg = 5.0`, що додає до функції втрат штраф пропорційно сумі квадратів значень у листках дерев. Більше значення цього параметра відповідає сильнішій регуляризації і робить модель консервативнішою у прогнозах. Емпірично визначене значення 5.0 забезпечує компроміс між точністю на навчальній вибірці та якістю узагальнення на тестовій. Додатково використовується параметр `min_data_in_leaf = 10`, що забороняє створення листків, що містять менше десяти прикладів, що також обмежує здатність моделі до запам'ятовування шумових патернів.

Стратегія зростання дерев визначається параметром `grow_policy = "Lossguide"`, що дозволяє листоорієнтоване зростання, при якому розщеплення першочергово застосовується до листків, що найбільше зменшують функцію втрат. Це робить дерева асиметричними, на відміну від класичної симетричної стратегії

CatBoost. Лістоорієнтоване зростання дозволяє моделі зосереджуватись на найбільш інформативних розщепленнях і часто демонструє кращу якість при середніх обсягах навчальних вибірок. Усі обчислення виконуються паралельно через параметр `thread_count = -1`, що використовує всі доступні CPU-ядра.

3.2.2 Реалізація Gradient Boosting

Модель Gradient Boosting реалізовано через клас `GradientBoostingClassifier` з бібліотеки `scikit-learn`. Навчальна вибірка формується аналогічно CatBoost з обсягом 1.1 мільйона прикладів і збагачена частка ПРК до приблизно 35-40%. На відміну від Random Forest, де всі дерева навчаються паралельно і незалежно, Gradient Boosting будує дерева послідовно. Кожне наступне дерево навчається на залишкових помилках попередніх. Така схема ефективніше використовує великі обсяги даних, оскільки кожне нове дерево виявляє все більш тонкі закономірності, які не були враховані попередніми моделями.

Відсутність вбудованого механізму ранньої зупинки з окремим валідаційним набором у `scikit-learn GradientBoostingClassifier` вирішується через параметри `n_iter_no_change = 20` та `validation_fraction = 0.1`. Алгоритм автоматично відкладає 10% тренувальної вибірки для внутрішньої валідації і зупиняється, коли відсутнє покращення протягом 20 ітерацій. Розмір `validation_fraction = 0.1` забезпечує достатньо велику валідаційну підмножину для статистично надійної оцінки метрик якості без істотного зменшення тренувальної вибірки.

Компенсація дисбалансу класів для Gradient Boosting реалізується через масив `sample_weight`, де кожному прикладу позитивного класу присвоюється вага `NATURAL_SPW ≈ 10.1`, а прикладам негативного класу є вага 1.0. На відміну від CatBoost, що використовує єдиний глобальний параметр `scale_pos_weight`, `scikit-learn` дозволяє задавати індивідуальні ваги для кожного прикладу, що відкриває можливості для більш складних схем зважування. У даній роботі використовується найпростіша схема рівних ваг у межах кожного класу.

Конфігурація моделі включає до 500 дерев рішень глибиною 9 рівнів. Верхня межа в 500 дерев є завідомо завищеною оцінкою, оскільки фактична кількість визначається механізмом ранньої зупинки і зазвичай становить від 250 до 400 дерев. Глибина дерев 9 рівнів є більшою порівняно з CatBoost і відображає різницю в стратегіях регуляризації між двома реалізаціями. CatBoost застосовує симетричні oblivious-дерева, що самі по собі є потужним регуляризатором, тоді як scikit-learn будує асиметричні дерева, де глибина 9 у поєднанні з стохастичними компонентами забезпечує оптимальний баланс.

Параметр `subsample = 0.8` вказує, що кожне дерево навчається на 80% навчальних прикладів, обраних випадково без повернення. Цей механізм був запропонований Фрідманом [22] як модифікація класичного Gradient Boosting і отримав назву Stochastic Gradient Boosting. Параметр `max_features = 0.8` обмежує кількість ознак, що розглядаються при виборі оптимального розбиття у кожному вузлі, до 80% від загальної кількості. Обидва ці параметри збільшують різноманітність дерев в ансамблі і зменшують кореляцію між ними, що покращує якість узагальнення.

3.2.3 Реалізація Random Forest

Модель Random Forest реалізовано через клас `RandomForestClassifier` з бібліотеки `scikit-learn`. На відміну від двох попередніх моделей, Random Forest використовує меншу навчальну вибірку з 500000 прикладів. Менший обсяг вибірки зумовлений особливостями алгоритму. Random Forest навчає всі дерева незалежно і зберігає повну структуру кожного дерева у пам'яті. При 600 деревах глибиною 15 рівнів розмір збереженої моделі може легко перевищувати 5 гігабайт. Для великих навчальних наборів це значний недолік, що обмежує практичне застосування.

Ансамбль складається з 600 дерев рішень, кожне з яких навчається на незалежній bootstrap вибірці. Вибір саме 600 дерев обумовлений компромісом між якістю прогнозу та обчислювальними витратами. Згідно з класичними результатами [9], помилка узагальнення Random Forest монотонно спадає зі

збільшенням кількості дерев і досягає асимптотичного значення, після якого додавання нових дерев не дає статистично значущого приросту якості. Bootstrap вибірка формується відбором із поверненням, тобто деякі навчальні приклади можуть зустрічатися в одному дереві кілька разів, тоді як інші взагалі не потрапляють до нього.

Параметр `max_depth = 15` обмежує максимальну глибину кожного дерева. Обмеження глибини є ефективним засобом регуляризації, що запобігає надмірній спеціалізації окремих дерев на шумах навчальної вибірки. Параметр `min_samples_leaf = 4` вимагає, щоб кожен листок дерева містив не менше чотирьох прикладів, що є додатковим засобом регуляризації. При виборі розбиття у кожному вузлі дерева розглядається лише приблизно 15 випадково обраних ознак з усіх 240, що суттєво збільшує різноманітність дерев і зменшує їхню кореляцію між собою.

Балансування класів у Random Forest реалізується через параметр `class_weight = "balanced_subsample"`, який присвоює прикладам міноритарного класу більшу вагу, обернено пропорційну його частоті. Цей режим відрізняється від стандартного тим, що ваги класів перераховуються для кожної bootstrap вибірки окремо на основі її фактичного розподілу класів, а не на основі всього тренувального набору. Такий підхід є більш адаптивним, оскільки враховує конкретний склад кожної підвибірки. Усі дерева навчаються паралельно через параметр `n_jobs = -1`, що використовує всі доступні процесорні ядра і суттєво знижує час навчання.

3.3 Реалізація ансамблевих моделей

Ансамблеві моделі є метамоделями, що поєднують декілька базових класифікаторів через зважене усереднення їхніх ймовірнісних виходів, воно ж у англійській літературі називається *soft voting*. На відміну від *hard voting*, де підраховуються голоси за конкретний клас, *soft voting* зберігає повну інформацію про ймовірнісний прогноз кожної компоненти, що є принципово важливим для

коректної роботи після калібрування. Ансамблеві моделі у даній роботі реалізовано через єдиний клас `EnsembleModel`, що інкапсулює всю необхідну функціональність. Збереження компонентів, обчислення зваженого прогнозу, калібрування та серіалізацію. Повний код класу `EnsembleModel` наведено у Додатку А.

Клас `EnsembleModel` виокремлено в окремий модуль `ensemble_model.py`, що є принципово важливим архітектурним рішенням з точки зору серіалізації. Коли Python зберігає об'єкт через `pickle`, він записує повний шлях до класу, включно з назвою модуля. Якщо клас знаходиться у головному скрипті, при завантаженні в іншому контексті виникає `AttributeError`. Виокремлення класу в окремий модуль з фіксованою назвою усуває цю проблему та гарантує коректне завантаження збережених ансамблів незалежно від способу запуску скрипту.

У роботі реалізовано три ансамблеві архітектури, що реалізують диверсифікацію через різні механізми. Підсумкове порівняння ансамблів наведено у табл. 3.2.

Таблиця 3.2 – Архітектура ансамблевих моделей

Ансамбль	Компоненти	ПРК у train	Ключова ідея
E1_Mixed	RF + GB + CB	35% для всіх	Диверсифікація алгоритмів. Різні inductive bias знижують кореляцію помилок
E2_CB_Bias	CB(10%) CB(50%) CB(90%)	+ + різний для кожного	Диверсифікація чутливості. Різне зібгачення формує різну агресивність прогнозу
E3_CB_Hyper	CB-shallow + CB-deep + CB-balanced	35% для всіх	Диверсифікація гіперпараметрів. Різна глибина та lr виявляють різні патерни

3.3.1 Ансамбль E1_Mixed

Перша ансамблева архітектура E1_Mixed реалізує найбільш класичну стратегію ансамблювання, а саме поєднання принципово різних алгоритмів машинного навчання. Теоретичне обґрунтування цього підходу полягає у тому, що Random Forest, Gradient Boosting та CatBoost належать до різних класів ансамблів і по-різному розкладають загальну похибку між систематичним відхиленням та дисперсією. Random Forest будує паралельний ансамбль із використанням беггінгу і мінімізує дисперсію за рахунок усереднення. Gradient Boosting та CatBoost будують послідовний ансамбль, де кожне наступне дерево явно компенсує помилки попередніх, що дозволяє ефективно зменшувати систематичне відхилення.

Для навчання E1_Mixed усі три базові моделі отримують спільну збагачену вибірку обсягом 800000 прикладів з цільовою часткою ПРК 35%. Обсяг 800000 є меншим порівняно з навчанням ізольованих базових моделей, що зумовлено компромісом між якістю кожного компонента і загальним часом навчання ансамблю. Конфігурація Random Forest у E1_Mixed є дещо легшою порівняно з ізольованою базовою моделлю та складає 400 дерев глибиною 12 замість 600 дерев глибиною 15. Це знижує час навчання та розмір збереженого pkl файлу. Конфігурація Gradient Boosting у E1_Mixed є близькою до ізольованої версії з 500 деревами глибиною 8. CatBoost у E1_Mixed навчається з тими самими ключовими параметрами, що й ізольована базова модель, з ранньою зупинкою через 150 ітерацій.

3.3.2 Ансамбль E2_CB_Bias

Друга ансамблева архітектура E2_CB_Bias реалізує принципово інший підхід до диверсифікації. Три версії одного і того самого алгоритму CatBoost навчаються на вибірках з різним рівнем збагачення ПРК прикладами. Ключовим теоретичним

обґрунтуванням цієї стратегії є те, що різний баланс класів у навчальній вибірці формує різну агресивність прогнозу моделі. Модель, навчена на вибірці з переважаючим міноритарним класом, схильна систематично передбачати наявність ПРК. Натомість, модель, навчена на вибірці з природним розподілом, є більш консервативною. Усереднення цих різних точок зору дозволяє формувати більш збалансований прогноз.

Кожна з трьох компонентних моделей навчається на окремій незалежно згенерованій вибірці обсягом 800000 прикладів. Перша компонента СВ-10 отримує вибірку з 10% ПРК-прикладів, що є близьким до модельного розподілу генератора. Друга компонента СВ-50 навчається на збалансованій вибірці з рівними частками класів. Третя компонента СВ-90 навчається на вибірці з переважаючою часткою ПРК у 90%. Параметр `scale_pos_weight` для кожної компоненти обчислюється від її власної тренувальної вибірки, що принципово відрізняє E2_CB_Bias від E1_Mixed та E3_CB_Hyper, де SPW однаковий для всіх компонентів. Ідентифікатори `random_seed` для трьох компонентів обираються виходячи з цільового відсотку ПРК: для СВ-10 `seed = 10`, для СВ-50 `seed = 50`, для СВ-90 `seed = 90`.

3.3.3 Ансамбль E3_CB_Hyper

Третя ансамблева архітектура E3_CB_Hyper реалізує диверсифікацію через різні конфігурації гіперпараметрів при незмінному балансі навчальних даних. Усі три компоненти навчаються на одній і тій самій спільно згенерованій вибірці з 800000 прикладів при цільовому рівні ПРК 35%. Спільна вибірка забезпечує, що різниця у поведінці компонентів зумовлена виключно їхніми гіперпараметрами. Для підвищення різноманітності кожен компонент використовує унікальний `random_seed`.

Перша компонента СВ-А налаштована як маленька швидка модель з глибиною дерев `depth = 5` та відносно великим темпом навчання `learning_rate = 0.10`. Друга компонента СВ-В є протилежністю СВ-А, глибокі дерева `depth = 9` з

повільним темпом навчання $\text{learning_rate} = 0.03$. Підвищена регуляризація $\text{l2_leaf_reg} = 8.0$ компенсує ризик перенавчання при великій глибині дерев. Третя компонента СВ-С є збалансованим середнім, де $\text{depth} = 7$, $\text{learning_rate} = 0.05$, симетричне зростання `SymmetricTree` та найвища регуляризація $\text{l2_leaf_reg} = 12.0$ у поєднанні з `subsampling subsample = 0.7`. Поєднання трьох компонентів з принципово різними гіперпараметрами забезпечує виявлення різних патернів у даних.

3.4 Калібрування ймовірностей

Калібрування ймовірностей є технікою машинного навчання, що коригує вихідні прогнози класифікатора таким чином, щоб числові значення ймовірностей достовірно відповідали реальній частоті настання подій. Класифікатор вважається добре відкаліброваним тоді, коли серед усіх прикладів, для яких модель повертає значення ймовірності близьке до p , частка прикладів, що дійсно належать до позитивного класу, також становить приблизно p . На жаль, більшість сучасних алгоритмів машинного навчання, особливо ансамблеві методи на основі дерев рішень, за замовчуванням не забезпечують такої відповідності, оскільки оптимізуються за критеріями точності класифікації, а не за якістю самих ймовірнісних оцінок.

Проблема некоректно відкаліброваних ймовірностей особливо гостро проявляється при дисбалансі класів та навчанні на збагачених вибірках [23, 24]. Коли навчальна вибірка містить штучно підвищену частку рідкісного класу, модель засвоює цю підвищену частку як апіорну ймовірність і свої числові виходи систематично зміщує у бік рідкісного класу. Якщо модель навчена на вибірці з 35% ПРК, вона схильна повертати значення ймовірності, що відповідають апіорній вірогідності 35%, а не реальній модельній вірогідності 9%. Це робить числові виходи моделі непридатними для коректного прийняття рішень і потребує процедури калібрування.

Для калібрування ймовірностей обрано ізотонічну регресію . Це непараметричний метод підбору монотонно зростаючої функції перетворення від некаліброваного виходу моделі до відкаліброваної ймовірності. Ізотонічна регресія вирішує задачу мінімізації суми квадратів відхилень між передбаченими ймовірностями та реальними частотами класів при обмеженні монотонності функції перетворення. Монотонність є природньою вимогою для калібрувальної функції. Якщо одна конфігурація отримала вищий некалібрований вихід ніж інша, після калібрування вона повинна отримати вищу або рівну відкалібровану ймовірність.

На відміну від альтернативного методу Платта, що підбирає сигмоїдну функцію перетворення через логістичну регресію [24], ізотонічна регресія не накладає жодних параметричних обмежень на форму функції перетворення. Це є суттєвою перевагою для моделей з нестандартним розподілом виходів, таким як CatBoost та Gradient Boosting у даній задачі. Алгоритм ізотонічної регресії будує кусково сталу монотонно зростаючу функцію шляхом об'єднання сусідніх порушень монотонності методом pool adjacent violators.

Технічна реалізація калібрування здійснюється через клас CalibratedClassifierCV із бібліотеки scikit-learn з параметрами `method = "isotonic"` та `cv = "prefit"`. Режим `prefit` є критично важливим для коректності процедури. Він сигналізує класу, що базова модель вже повністю навчена і калібрувальний шар повинен навчатися лише на переданій вибірці без виконання додаткової крос-валідації. Для кожної базової моделі та для кожного компонента кожного ансамблю калібрування виконується незалежно на одній і тій самій модельній вибірці з 8000 прикладів. Незалежне калібрування кожного компонента ансамблю забезпечує, що всі компоненти мають коректно відкалібровані виходи, і їхнє середнє є природньо відкаліброваним ансамблевим прогнозом без необхідності у повторному калібруванні на рівні ансамблю.

3.5 Вибір порогу класифікації

Після калібрування числові виходи моделей стають коректно відкаліброваними ймовірностями, що відповідають модельному розподілу класів. Проте для прийняття бінарних рішень про класифікацію кластера як такого, що знаходиться у стані ПРК, необхідно обрати поріг класифікації $\tau \in [0, 1]$. Вибір порогу є ключовим етапом, що визначає співвідношення між двома типами помилок хибно негативними прогнозами та хибно позитивними прогнозами. У даній роботі реалізовано три методи вибору порогу, що відповідають різним стратегіям компромісу між цими типами помилок.

Стандартний поріг 0.5 є неприйнятним навіть після калібрування при модельному рівні ПРК 9%. При порозі 0.5 модель передбачатиме ПРК лише для конфігурацій з дуже сильними ознаками ПРК, що призводить до низького Recall і пропуску значної частини реальних випадків ПРК. Оптимальний поріг при дисбалансі класів є значно нижчим за 0.5 і наближається до модельної частки рідкісного класу, тобто до значень порядку 0.09-0.15.

Метод Юдена є рекомендованим та основним методом вибору порогу у даній роботі. Статистика Юдена була запропонована В. Дж. Юденом у 1950 році для оцінки якості діагностичних тестів [25] і визначається як $J = \text{TPR} - \text{FPR}$, що еквівалентно $J = \text{Sensitivity} + \text{Specificity} - 1$. Статистика приймає значення в діапазоні від -1 до 1 , де $J = 1$ відповідає ідеальному класифікатору. Знайдений поріг, що максимізує J , забезпечує найкращий загальний баланс між чутливістю та специфічністю. Принциповою особливістю методу Юдена є те, що він надає однакову вагу обом типам помилок при визначенні оптимального порогу, що робить його неупередженим інструментом.

Метод MaxF1 обирає поріг, що максимізує F1 score на PR-кривій. F1-score визначається як гармонічне середнє точності та повноти і є чутливим до меншого з двох показників. Це означає, що F1 score штрафує моделі, які демонструють високу повноту ціною низької точності або навпаки, і заохочує моделі зі збалансованою

якістю прогнозу. Поріг MaxF1 зазвичай є вищим за поріг Юдена, оскільки оптимізація F1 схиляється до більш консервативних прогнозів, що мінімізують хибно позитивні помилки на шкоду Recall.

Метод High-Recall [26] знаходить мінімальний поріг, при якому Recall досягає щонайменше 0.80, тобто система виявляє не менше 80% реальних ПРК кластерів. Формально цей метод реалізує задачу умовної оптимізації. Серед усіх порогів τ , що задовольняють обмеження $\text{Recall}(\tau) \geq 0.80$, обирається той, що максимізує Precision. Вибір саме 80% як цільового рівня Recall обумовлений компромісом між надійністю виявлення критичних станів та практичною прийнятністю рівня хибних тривог. Метод High-Recall є найбільш консервативним щодо пропуску ПРК і рекомендований для сценаріїв, де вартість пропущеної ПРК суттєво перевищує вартість хибної тривоги.

Усі три знайдені пороги для кожної базової моделі та кожного ансамблю зберігаються у форматі JSON поряд із відповідними pkl файлами моделей. JSON файл містить рекомендований метод, а також конкретні числові значення порогів і відповідних метрик. Це дозволяє при роботі системи обирати відповідний поріг залежно від контексту застосування без необхідності у повторному пошуку. За замовчуванням при прогнозуванні через веб сервіс використовується поріг Юдена як найбільш збалансований.

Важливим аспектом пошуку оптимального порогу є те, що він виконується на тестовій вибірці після калібрування, а не до. Якщо шукати поріг на некаліброваних виходах, знайдений поріг може не переноситися на відкалібровані виходи через зміну розподілу значень ймовірностей. Крім того, тестова вибірка, на якій виконується пошук порогу, є окремою від калібрувальної вибірки. Використання однієї вибірки і для калібрування, і для пошуку порогу призвело б до оптимістично зміщених оцінок.

3.6 Веб-інтерфейс системи

Веб-інтерфейс системи реалізовано на основі фреймворку Django і призначений для надання користувачам інтерактивного засобу побудови довільних топологій кластера та отримання прогнозу ймовірності ПРК у реальному часі. Інтерфейс реалізує сценарії використання архітектора кластерів, описані у Розділі 2 і відображені на діаграмі варіантів використання. Загальний вигляд основної сторінки веб-інтерфейсу наведено на рис. 3.2.

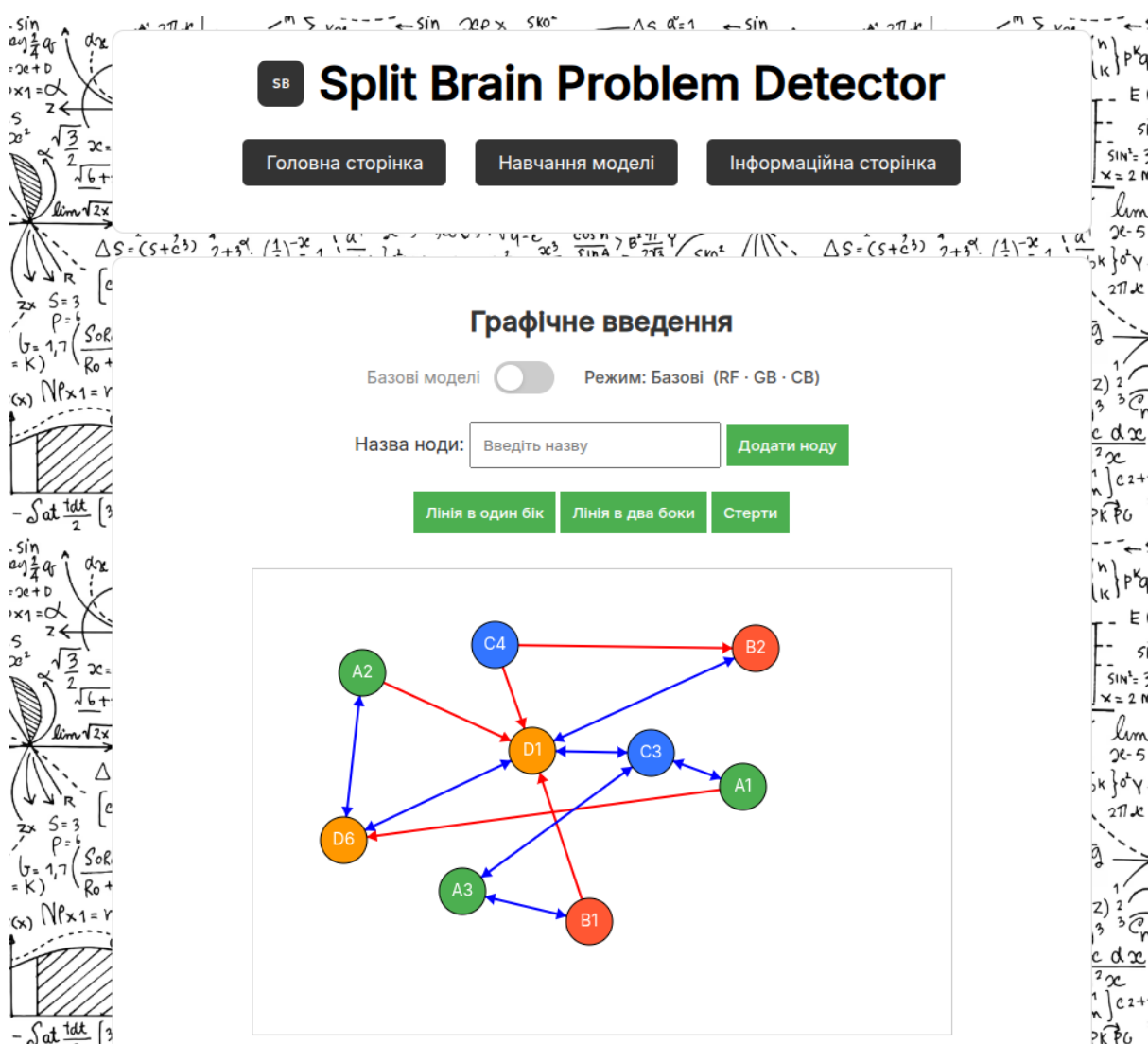


Рисунок 3.2 – Користувацький інтерфейс сторінки "Графічне завдання"

Структура веб-застосунку включає чотири основні сторінки. Головна сторінка надає короткий опис проєкту, його наукового підґрунтя та інструкції з початку роботи. Сторінка "Графічне завдання" є основним функціональним екраном, на якому користувач взаємодіє з топологією кластера. Сторінка "Навчання моделі" надає інтерфейс для запуску процесу донавчання моделей через веб форму, що корисно у випадку, коли користувач має власну калібрувальну вибірку. Інформаційна сторінка містить розширене пояснення природи ПРК, опис методики роботи системи та посилання на наукові публікації.

Основна функціональність зосереджена на сторінці "Графічне завдання". На цій сторінці реалізовано інтерактивний графічний редактор топології кластера на основі HTML5 Canvas. Користувач додає ноди до кластера шляхом введення їхніх назв у текстове поле. Назва повинна відповідати формату XY, де X — велика латинська літера, що позначає тип мікросервісу, а Y — цифра від 1 до 9, що є порядковим номером ноди відповідного типу. Після підтвердження ноди вона з'являється на полотні як коло з відповідним кольоровим маркуванням типу. Ноди можна переміщувати по полотну за допомогою миші, що дозволяє користувачеві впорядкувати топологію відповідно до його концептуальної моделі.

Встановлення зв'язків між нодами реалізовано через перемикач режимів. Користувач обирає тип зв'язку, односторонній або двосторонній, і послідовно натискає на дві ноди для створення відповідного зв'язку. Двосторонні зв'язки відображаються блакитними лініями, односторонні є стрілками червоного кольору від однієї ноди до іншої. У контексті прогнозування ПРК використовуються лише двосторонні зв'язки, оскільки саме вони визначають структуру комунікації мікросервісів. Інтерфейс також надає кнопку видалення нод та зв'язків для коригування топології.

Паралельно з графічним полотном на сторінці відображається матриця доступності. Це табличне представлення поточної топології у вигляді бінарної матриці $N \times N$ з іменами нод у заголовках рядків і стовпців. Матриця доступності оновлюється автоматично при кожній зміні топології і служить додатковим

засобом верифікації структури кластера. Після завершення побудови топології користувач натискає кнопку "Відправити", що ініціює http post запит до сервера з поточною конфігурацією кластера.

На стороні сервера обробка запиту проходить через декілька етапів. Контролер Django приймає JSON дані запиту і викликає функцію preprocess для перетворення матриці суміжності у вектор ознак фіксованої розмірності 240. Сформований вектор передається до модуля інференсу, який послідовно викликає метод predict_proba для обраного типу моделей. Усі моделі завантажуються з pickle файлів при першому запиті і кешуються у пам'яті процесу для прискорення наступних викликів. Результати прогнозу серіалізуються у JSON і повертаються клієнту як відповідь на http запит.

На клієнтській стороні отримані результати відображаються у таблиці на правій частині сторінки. Кожен рядок таблиці містить назву моделі та її прогноз ймовірності ПРК у відсотках. Окрім самих прогнозів, інтерфейс відображає поточний поріг класифікації, що є порогом Юдена, а також узагальнену рекомендацію щодо стабільності конфігурації. Така структура виведення дозволяє користувачеві швидко оцінити загальну тенденцію і отримати конкретні числові значення для подальшого аналізу.

Реалізація http обробки використовує асинхронний підхід через JavaScript Fetch API. Запит на отримання прогнозу не блокує користувацький інтерфейс, що дозволяє користувачеві продовжувати взаємодію з топологією під час обробки запиту. Під час очікування відповіді інтерфейс відображає індикатор завантаження, що інформує користувача про активний стан обробки. Загальний час відгуку системи на прогнозуючий запит становить від 50 до 200 мілісекунд для гарячого стану системи і може досягати кількох секунд при холодному запуску, коли потребується завантаження великих pickle файлів з диска.

Веб-інтерфейс реалізує також додаткову функціональність валідації введених даних на клієнтській стороні. Перед відправленням запиту JavaScript перевіряє коректність формату назв нод, наявність хоча б однієї ноди кожного типу

у кластері та консистентність матриці суміжності. У разі виявлення помилок користувач отримує відповідне повідомлення без необхідності у мережевому раундтрипі до сервера. Така валідація покращує користувацький досвід і знижує навантаження на серверну частину системи.

Висновки до розділу 3

У третьому розділі представлено повну програмну реалізацію системи. Обрано та обґрунтовано технологічний стек. Детально розкрито реалізацію трьох базових моделей з повним переліком гіперпараметрів. Реалізовано три ансамблеві архітектури з різними стратегіями диверсифікації. Усі ансамблі реалізують soft voting через єдиний клас EnsembleModel, повний код якого наведено у Додатку А.

Реалізовано процедуру ізотонічного калібрування ймовірностей через CalibratedClassifierCV з режимом prefit, що забезпечує коректне використання окремої калібрувальної вибірки. Описано три методи пошуку оптимального порогу класифікації. Розроблено веб-інтерфейс системи на основі Django з інтерактивним графічним редактором топології кластера, що дозволяє користувачам у режимі реального часу будувати довільні конфігурації та отримувати прогнози ймовірності ПРК від усіх шести моделей системи.

4 ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ

4.1 Методика експериментів

Експериментальне дослідження реалізованих моделей машинного навчання проводилось з метою кількісної оцінки якості бінарної класифікації конфігурацій кластерів за станом ПРК. Усі експерименти виконувались на синтетичних даних, згенерованих за методикою, описаною у Розділі 2. Метою експериментів є порівняння трьох базових моделей та трьох ансамблевих моделей за повним набором метрик якості, а також аналіз ефективності процедури калібрування ймовірностей та трьох методів вибору порогу класифікації.

Усі експерименти виконувались з фіксованим значенням генератора псевдовипадкових чисел `random_seed = 42`, що забезпечує повну відтворюваність результатів при повторному запуску. Інші значення `seed` не використовувались, тому всі наведені числові результати є реалізацією одного експерименту і не оцінюють варіативність метрик між запусками з різними початковими станами. Це є обмеженням дослідження, що детально обговорюється у підрозділі 4.6.

Тестова вибірка обсягом 20000 прикладів є фіксованою для всіх моделей і генерується першою у процесі підготовки даних, перед навчанням будь-якої моделі. Розподіл класів у тестовій вибірці відповідає модельному, що становить 9.02% позитивних прикладів, що практично співпадає з очікуваним значенням 9% для обраних параметрів генератора Ердьош-Реньї. Калібрувальна вибірка обсягом 8000 прикладів є окремою від тестової і використовується виключно для процедури ізотонічного калібрування ймовірностей. Жодна з цих вибірок не задіюється у процесі навчання базових моделей.

Експерименти проводились на персональному комп'ютері з процесором, що має 16 логічних ядер, та 32 ГБ оперативної пам'яті. Загальний час навчання всіх шести моделей з нуля складає приблизно 4-5 годин. Оцінка моделей на тестовій вибірці виконується значно швидше. Середній час інференсу одного прикладу

становить від 0.055 мс для Gradient Boosting до 0.221 мс для ансамблю E2_CB_Bias, що дозволяє оцінити повну тестову вибірку у межах кількох секунд для кожної моделі.

Усі експерименти проводились у два етапи. Спочатку аналізується поведінка моделей до калібрування ймовірностей при стандартному порозі 0.5, а потім вже після калібрування з оптимальними порогами, обраними одним із трьох методів. Такий двоетапний аналіз дозволяє ізолювати ефекти двох ключових процедур постобробки і кількісно оцінити їхній внесок у фінальну якість моделей.

4.2 Результати моделей до калібрування

Першим етапом експериментального дослідження є оцінка моделей у їхньому сирому стані, тобто без застосування процедури калібрування ймовірностей та з використанням стандартного порогу класифікації 0.5. Така оцінка є важливою з двох причин. По-перше, вона демонструє фактичну поведінку моделей одразу після навчання, що дозволяє виявити систематичні зміщення, спричинені дисбалансом класів у навчальній вибірці. По-друге, отримані результати створюють базову точку відліку для порівняння з результатами після калібрування у підрозділі 4.3.

Підсумкові результати оцінки шести моделей до калібрування при порозі $\tau = 0.5$ наведено у табл. 4.1. У таблиці представлено ключові метрики. Усі значення обчислено на спільній тестовій вибірці з 20000 прикладів.

Таблиця 4.1 – Метрики моделей до калібрування при порозі $\tau = 0.5$

Модель	ROC-AUC	PR-AUC	Precision	Recall	F1	FPR	ECE
CatBoost	0.7972	0.3800	0.1054	0.9956	0.1906	0.8378	0.6401
Gradient Boosting	0.7761	0.3519	0.1027	0.9983	0.1862	0.8648	0.6482
Random Forest	0.6619	0.2370	0.1760	0.2733	0.2141	0.1268	0.3484
E1_Mixed	0.7208	0.2597	0.0980	0.9989	0.1785	0.9114	0.6346
E2_CB_Bias	0.7662	0.3483	0.1697	0.7445	0.2765	0.3610	0.3334
E3_CB_Hyper	0.7872	0.3484	0.0968	0.9989	0.1765	0.9242	0.7619

Аналіз отриманих результатів виявляє декілька принципових закономірностей. Перша і найбільш важлива це те, що значення Recall для більшості моделей наближається до одиниці, що на перший погляд може здатися надзвичайно високим результатом. Проте паралельно з цим спостерігається катастрофічно високий FPR. Це означає, що моделі практично всі негативні приклади також класифікують як позитивні. Така поведінка робить ці моделі практично некорисними при стандартному порозі.

Природа цього явища пов'язана з тим, що моделі навчались на штучно збагачених тренувальних вибірках з часткою ПРК приблизно 35%, тоді як модельна частка у тестовій вибірці становить лише 9.02%. Моделі засвоїли цей вищий рівень як апріорну ймовірність позитивного класу і систематично переоцінюють ймовірність ПРК для всіх вхідних прикладів. Як наслідок, значна частка прикладів отримує прогнозовану ймовірність вище 0.5, незалежно від їхнього реального класу. Це підтверджується значеннями метрики ECE, що для CatBoost, Gradient Boosting, E1_Mixed та E3_CB_Hyper перевищують 0.6, що відповідає принциповому неузгодженню між прогнозованими і реальними частотами.

Random Forest демонструє принципово відмінну поведінку. Recall становить лише 0.2733, тоді як FPR значно нижчий, а ECE є помітно меншим. Це пояснюється

особливістю реалізації Random Forest у scikit-learn, де режим `class_weight = "balanced_subsample"` перераховує ваги класів для кожної bootstrap-вибірки окремо, що знижує систематичне зміщення моделі у бік збагаченого розподілу. Проте ця стійкість до зміщення досягається ціною низького Recall. Модель пропускає переважну більшість реальних випадків ПРК з $\text{false negative rate} = 0.7267$.

Виключенням з цієї картини серед ансамблів є E2_CB_Bias, що демонструє більш збалансовану поведінку. Це є наслідком архітектурної особливості ансамблю, де три компоненти CB-10, CB-50 та CB-90 навчались на вибірках з різними рівнями збагачення, і середній прогноз виявляється менш зміщеним, ніж у моделей, навчених на єдиній збагаченій вибірці.

Незважаючи на згадані проблеми, показники ROC-AUC та PR-AUC, що не залежать від конкретного порогу класифікації, свідчать про наявність у моделей корисної розрізнявальної здатності. CatBoost демонструє найвищий ROC-AUC та PR-AUC, що в 4.2 рази вище за базовий рівень випадкового класифікатора. Це означає, що моделі дійсно навчилися розрізняти конфігурації різних класів, але вихідні ймовірності потребують коригування через калібрування для практичного використання. PR-AUC від 0.24 до 0.38 у задачі з модельною часткою позитивного класу 9% є помітним перевищенням базового рівня, що підтверджує доцільність подальшої роботи з цими моделями після належної постобробки.

Графічне представлення некоректності калібрування ймовірностей до постобробки наведено на рис. 4.1, що містить діаграми надійності всіх шести моделей. На діаграмах суцільна лінія відображає залежність фактичної частоти позитивного класу від прогнозованої моделлю ймовірності, а пунктирна діагональ відповідає ідеальному калібруванню. Розмір точок пропорційний логарифму кількості прикладів у відповідному біні. Чим далі суцільна лінія від діагоналі, тим гіршим є калібрування моделі.

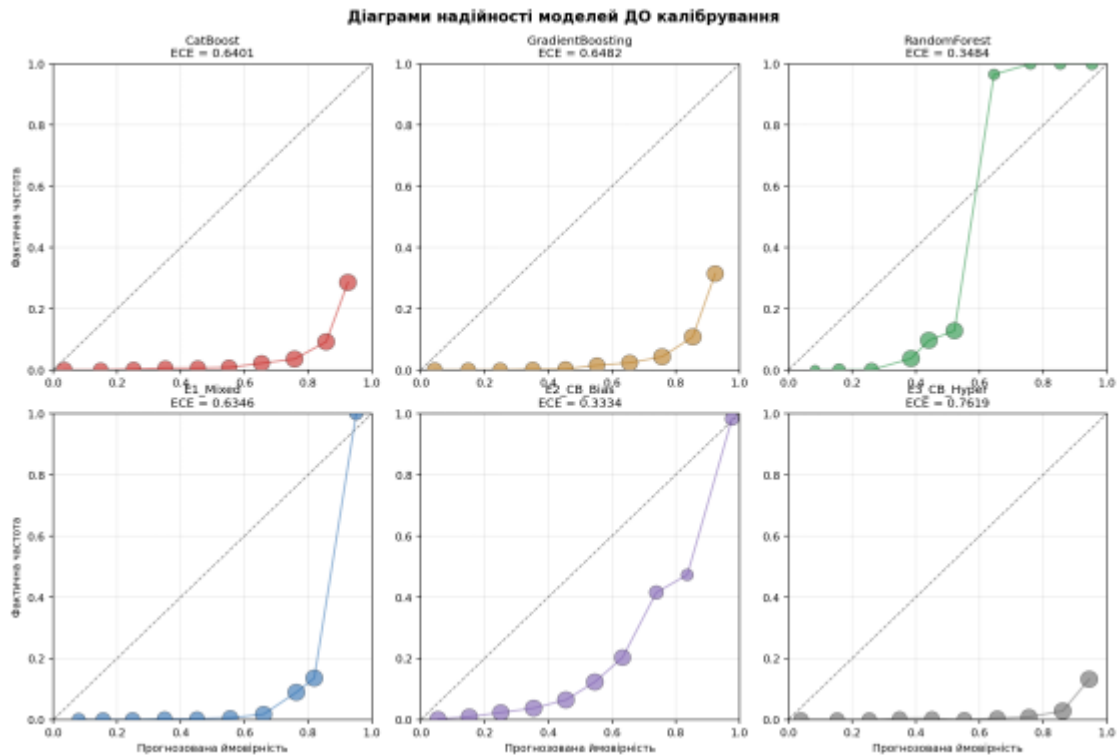


Рисунок 4.1 – Діаграми надійності моделей до калібрування

Як можна побачити з рисунка, для більшості моделей суцільна лінія знаходиться значно нижче діагоналі у середньому діапазоні ймовірностей, що свідчить про систематичне завищення прогнозованої ймовірності щодо реальної частоти.

4.3 Результати моделей після калібрування

Другим етапом експериментального дослідження є оцінка моделей після застосування процедури ізотонічного калібрування та вибору оптимального порогу методом Юдена. Калібрування проводиться на окремій вибірці з 8000 прикладів з модельним розподілом класів, а пошук порогу на тестовій вибірці після застосування калібрування. Метою цього етапу є демонстрація ефективності калібрування у виправленні систематичного зміщення вихідних ймовірностей та оцінка фінальної якості моделей у експлуатаційному режимі.

Підсумкові результати оцінки шести моделей після калібрування при оптимальних порогах методу Юдена наведено у табл. 4.2. У таблиці окремо вказано значення обраного порогу для кожної моделі, оскільки воно є індивідуальним і залежить від форми розподілу ймовірностей конкретної моделі.

Таблиця 4.2 – Метрики моделей після калібрування при порозі Юдена

Модель	Поріг τ	Precision	Recall	F1	FPR	ECE	Brier
CatBoost	0.0957	0.1801	0.7783	0.2925	0.3513	0.0030	0.0683
Gradient Boosting	0.0955	0.1835	0.6907	0.2900	0.3047	0.0052	0.0698
Random Forest	0.0899	0.1267	0.7389	0.2163	0.5051	0.0035	0.0744
E1_Mixed	0.1034	0.1763	0.7195	0.2833	0.3332	0.0136	0.0708
E2_CB_Bias	0.1031	0.1731	0.7278	0.2797	0.3447	0.0084	0.0698
E3_CB_Hyper	0.1137	0.1916	0.7029	0.3012	0.2940	0.0057	0.0698

Найбільш вражаючим результатом є радикальне зниження ECE для всіх моделей. Для CatBoost це з 0.6401 до 0.0030, а зменшення у 213 разів. Усі моделі після калібрування демонструють ECE менше 0.014, що свідчить про коректну відповідність прогнозованих ймовірностей реальним частотам класів. Аналогічну тенденцію спостерігається для метрики Brier score. Для CatBoost вона знижується з 0.5198 до 0.0683, для Gradient Boosting з 0.5206 до 0.0698, що відповідає покращенню у 7.5 разів.

Слід зазначити, що показник ROC-AUC після калібрування практично не змінюється. Для CatBoost він становить 0.7961 порівняно з 0.7972 до калібрування, для Gradient Boosting 0.7750 проти 0.7761, для Random Forest 0.6611 проти 0.6619. Відсутність змін ROC-AUC є очікуваним результатом, оскільки ізотонічна регресія є монотонним перетворенням, що не змінює порядку прогнозів моделі і, отже, не впливає на ROC-криву. Незначна різниця між значеннями до і після калібрування

пояснюється числовими артефактами обробки граничних значень у ізотонічній регресії.

Значення оптимальних порогів за методом Юдена для всіх моделей знаходиться у вузькому діапазоні від 0.0899 до 0.1137, що практично співпадає з модельною часткою позитивного класу 9%. Це є закономірним результатом теоретичних властивостей задач бінарної класифікації при дисбалансі класів. Оптимальний поріг наближається до частки позитивного класу при коректному калібруванні. Цей факт додатково підтверджує якість виконаного калібрування і обґрунтовує практичну значущість процедури.

Метрики Precision, Recall та F1 score після калібрування набувають практично прийнятних значень. CatBoost демонструє Recall 0.7783 при Precision 0.1801 та F1 0.2925, що означає виявлення приблизно 78% реальних випадків ПРК при тому, що з кожних шести позитивних прогнозів моделі лише один є істинно позитивним. Це є типовим компромісом для задач з сильним дисбалансом класів, при низькій модельній частці позитивного класу 9% досягти одночасно високих Precision і Recall є фундаментально складно.

Графічне підтвердження якості калібрування представлено на рис. 4.2, що містить діаграми надійності тих самих шести моделей після постобробки.

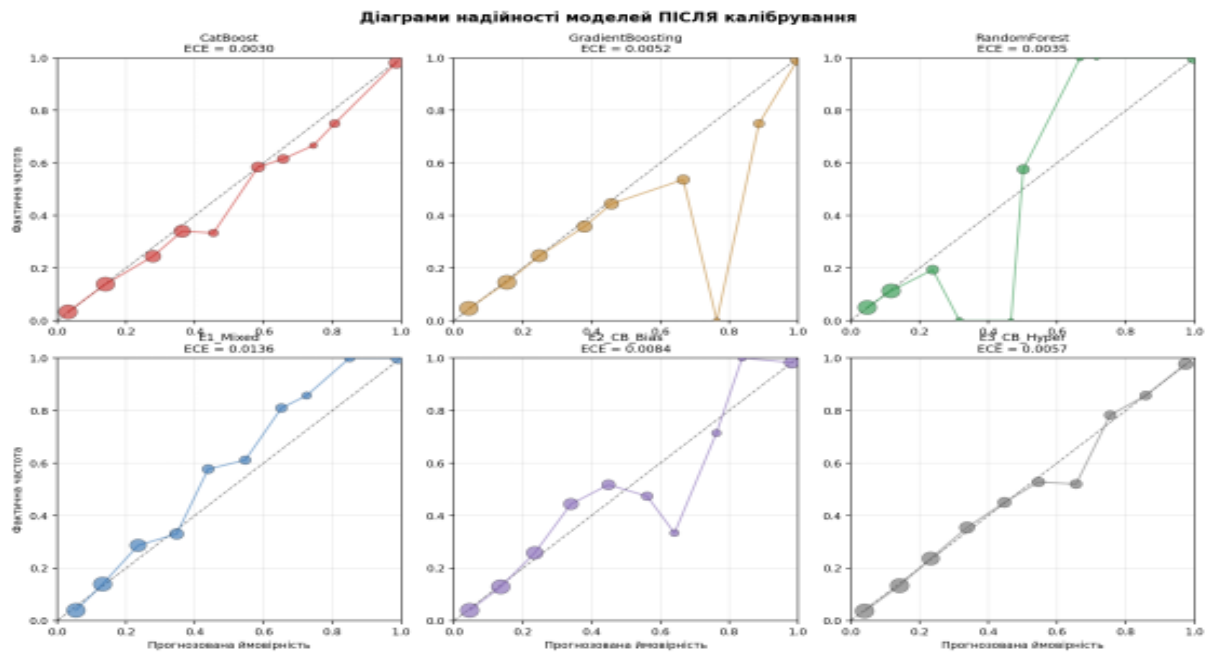


Рисунок 4.2 – Діаграми надійності моделей після калібрування

Порівняно з рис. 4.1, де суцільні лінії розташовувались далеко від діагоналі, на рис. 4.2 спостерігається близьке слідування діагоналі для всіх моделей у діапазонах прогнозованих ймовірностей з достатньою кількістю прикладів. Деяке відхилення у бінах з малою кількістю прикладів є природним наслідком статистичної невизначеності при малих обсягах даних і не впливає на загальну якість калібрування.

4.4 Порівняння методів вибору порогу

Цей підрозділ присвячено порівнянню трьох методів вибору порогу класифікації, реалізованих у системі. Кожен метод відповідає різній стратегії компромісу між хибно негативними та хибно позитивними помилками і є придатним для різних сценаріїв застосування системи.

Підсумкові результати застосування трьох методів вибору порогу для всіх шести моделей наведено у табл. 4.3. Для кожного методу вказано знайдене значення порогу та відповідні ключові метрики.

Таблиця 4.3 – Порівняння методів вибору порогу класифікації

Модель	Юден τ	TPR	FPR	MaxF 1 τ	F1	HR τ	Recall	Precision (HR)
CatBoost	0.095 7	0.793 2	0.364 5	0.1843	0.360 7	0.088 8	0.817 1	0.1697
Gradient Boosting	0.095 5	0.690 7	0.304 7	0.1824	0.323 8	0.081 1	0.808 8	0.1580
Random Forest	0.089 9	0.781 6	0.543 1	0.1042	0.216 3	0.066 1	0.868 6	0.1156
E1_Mixed	0.103 4	0.719 5	0.333 2	0.1437	0.315 5	0.089 6	0.801 0	0.1585
E2_CB_Bias	0.103 1	0.727 8	0.343 5	0.1876	0.325 6	0.088 1	0.801 0	0.1553
E3_CB_Hyper	0.113 7	0.704 5	0.294 5	0.1710	0.337 9	0.090 2	0.800 4	0.1660

Аналіз отриманих результатів виявляє декілька закономірностей у роботі різних методів. Метод Юдена видає значення порогів у діапазоні від 0.0899 до 0.1137, що є найнижчими серед усіх методів. Відповідні значення TPR коливаються від 0.6907 до 0.7932, а FPR від 0.2940 до 0.5431. Серед усіх моделей найвище значення J-статистики 0.4287 досягається CatBoost, що ставить її на перше місце за сукупним балансом між чутливістю та специфічністю. Ансамбль E3_CB_Hyper має близьке значення $J = 0.41$, що є другим результатом.

Метод MaxF1 видає вищі пороги і відповідно більш консервативні прогнози. F1 score знаходиться у діапазоні від 0.2163 до 0.3607. CatBoost знову демонструє найвищий результат за цією метрикою. Збільшення порогу порівняно з методом Юдена відображає особливість F1 score. Ця метрика штрафувє хибно позитивні прогнози сильніше, ніж стандартний баланс TPR-FPR, що зрушує оптимальний поріг у бік більшої консервативності.

Метод High-Recall видає найнижчі пороги, оскільки його цільова функція вимагає виявлення не менше 80% реальних випадків ПРК. У відповідь на цю

вимогу метод суттєво знижує поріг, що дозволяє моделі позначати більше прикладів як позитивні. Як наслідок, Recall дійсно перевищує 0.80 для всіх моделей, однак Precision залишається низьким. Це означає, що при використанні методу High-Recall система генеруватиме значну кількість хибних тривог, що може бути виправданим у сценаріях, де вартість пропуску критичного стану суттєво перевищує вартість додаткової перевірки.

Загальна закономірність полягає у тому, що жоден метод не є універсально кращим. Кожен метод оптимізує власний критерій якості, і вибір конкретного методу залежить від практичних вимог сценарію використання. Метод Юдена є рекомендованим за замовчуванням як такий, що забезпечує найкращий загальний баланс між двома типами помилок. Метод MaxF1 є придатним для сценаріїв, де модель має продемонструвати збалансовану якість прогнозу для звітності. Метод High-Recall є придатним для сценаріїв з підвищеними вимогами до повноти виявлення критичних станів.

4.5 Порівняльний аналіз моделей

Останнім етапом експериментального дослідження є комплексний порівняльний аналіз усіх шести моделей за повним набором метрик. Метою цього аналізу є виявлення сильних та слабких сторін кожної моделі, а також обґрунтування рекомендацій щодо їхнього застосування у різних сценаріях. Графічне порівняння моделей за чотирма ключовими метриками наведено на рис. 4.3.

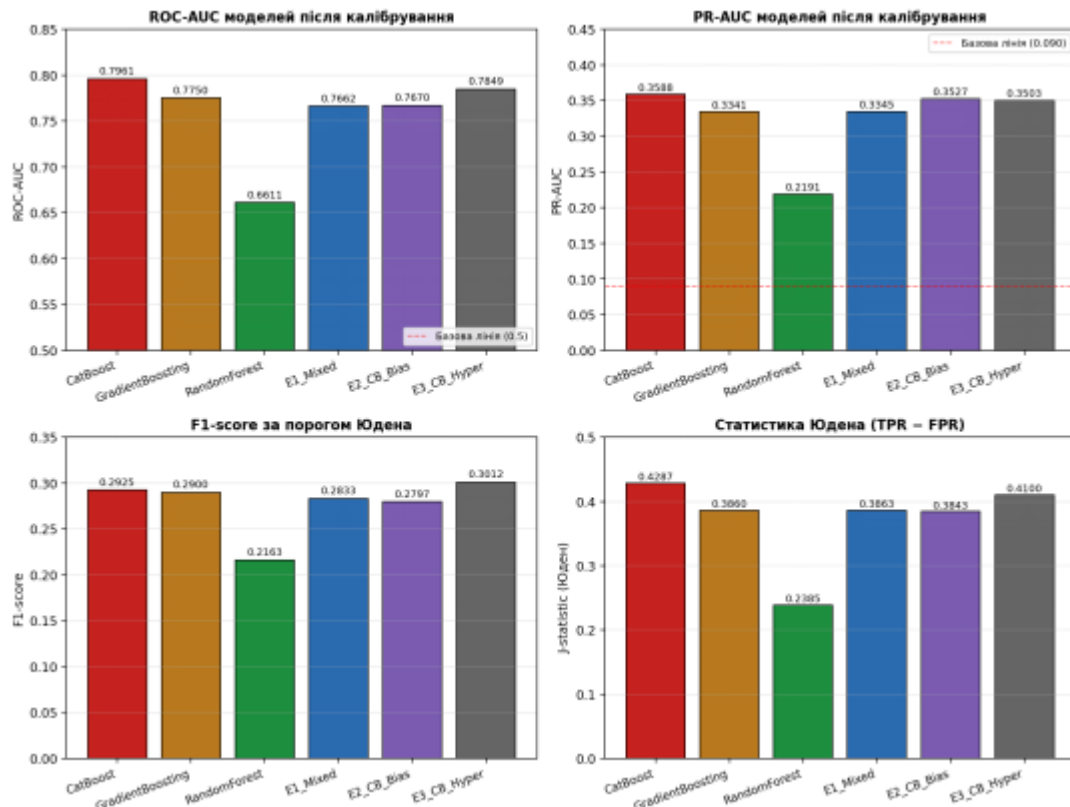


Рисунок 4.3 – Порівняння моделей за ключовими метриками

Найбільш чітко за всіма метриками виділяється модель CatBoost. Найкращий серед усіх моделей ROC-AUC 0.7961, другий результат за PR-AUC 0.3588, F1-score найвищий зі значенням 0.3607 при порозі MaxF1 та статистика Юдена 0.4287 найвища. Це робить CatBoost однією з найкращих моделей за всіма ключовими метриками і свідчить про високу ефективність обраних архітектурних рішень, а саме симетричні дерева, впорядкований бустинг, ранню зупинку та коректне налаштування `scale_pos_weight` через константу `NATURAL_SPW`.

Серед ансамблевих архітектур найкращі результати демонструє E3_CB_Hyper. Цей ансамбль є другим за всіма метриками після CatBoost і єдиним ансамблем, що наближається до якості найкращої базової моделі. Архітектура E3_CB_Hyper, що поєднує три CatBoost-компоненти з різними значеннями глибини, темпу навчання та регуляризації, виявляється успішною стратегією диверсифікації. Її успіх підтверджує доцільність диверсифікації через гіперпараметри у задачах з обмеженою кількістю принципово різних алгоритмів.

Ансамбль E2_CB_Bias має дещо нижчі показники, що тим не менш дещо вище за більшість компонентів. Хоча його PR-AUC 0.3527 є фактично другим серед усіх моделей після CatBoost, статистика Юдена нижча за E3_CB_Hyper, що пояснюється тим, що ця архітектура спеціалізується на ранжуванні позитивних випадків, але дещо гірше встановлює оптимальний поріг.

Ансамбль E1_Mixed, що поєднує принципово різні алгоритми, демонструє результати, що знаходяться на нижній межі діапазону. Очікувана перевага диверсифікації через різні алгоритми у даному випадку не реалізується повною мірою через те, що Random Forest, як один з трьох компонентів, має суттєво гіршу індивідуальну якість, що знижує загальний результат ансамблю. Це є важливим методологічним висновком. Ансамблеве усереднення є ефективним лише тоді, коли всі компоненти мають порівнянну якість.

Окремим висновком експериментального дослідження є те, що жоден з ансамблів у даній конфігурації не перевершує найкращу базову модель CatBoost за більшістю метрик. Перевага ансамблів традиційно асоціюється з можливістю компенсувати помилки окремих компонентів, але у даній задачі CatBoost виявився настільки сильною базовою моделлю, що додавання інших компонентів швидше додає шуму, ніж покращує загальну якість. Цей результат не суперечить теоретичним основам ансамблевого навчання, а свідчить про специфіку конкретної задачі. У задачах з менш контрастними базовими моделями ансамблі можуть демонструвати помітну перевагу.

Random Forest у даному дослідженні демонструє найнижчу якість серед усіх моделей. Однак це не означає, що алгоритм є непридатним для задачі, адже при іншому розмірі навчальної вибірки, іншому форматі стратегії балансування класів та обмеженій глибині дерев, можливо досягти кращих результатів. Включення Random Forest у це дослідження виправдане через його простоту, інтерпретованість та широку популярність у промислових застосуваннях.

Підсумовуючи результати експериментального дослідження, можна стверджувати, що усі шість моделей після калібрування демонструють корисну

розрізнявальну здатність для задачі бінарної класифікації конфігурацій кластерів за станом ПРК. Найкращі результати демонструє CatBoost, за яким слідує ансамбль E3_CB_Hyper. У реальних сценаріях використання системи рекомендується застосовувати саме ці дві моделі, з вибором конкретної залежно від компромісу між обчислювальною вартістю та потенційною стабільністю прогнозу.

4.6 Обмеження дослідження

Експериментальне дослідження, проведене у даному розділі, має ряд обмежень, які слід враховувати при інтерпретації отриманих результатів та при плануванні подальшого розвитку системи. У цьому підрозділі ці обмеження систематизовано і обговорено.

Першим і найбільш суттєвим обмеженням є використання синтетично згенерованих даних замість реальних топологій промислових кластерів. Дані генерувались за моделлю Ердьош-Реньї з фіксованим параметром $p_{\text{link}} = 0.7$, що накладає певні припущення на структуру топологій, а саме незалежність зв'язків між парами вузлів, рівномірний розподіл розмірів кластера у діапазоні від 4 до 15 нод, рівноймовірний вибір типів вузлів. У реальних промислових кластерах ці припущення можуть не виконуватися, адже зв'язки між нодами часто корелюють через географію дата центрів, розміри кластерів можуть мати інший розподіл, а типи вузлів можуть мати нерівномірне представлення. Отже, прогноз поведінки моделі на реальних даних потребує окремої валідації, що не виконувалось у даній роботі через відсутність публічно доступних реальних даних.

Другим обмеженням є використання єдиного значення `random_seed = 42` для всіх експериментів. Це означає, що всі наведені числові результати є реалізацією одного експерименту і не оцінюють варіативність метрик між запусками з різними початковими станами генератора псевдовипадкових чисел. Як наслідок, неможливо встановити, чи є невеликі різниці у метриках між моделями статистично значущими або знаходяться у межах випадкових варіацій. Для

надійнішої оцінки потрібно виконати декілька повторів експерименту з різними seeds і обчислити середні значення з довірчими інтервалами, що є природним напрямом подальшого розширення дослідження.

Третім обмеженням є фіксований діапазон розмірів кластера від 4 до 15 вузлів. Це обмеження було обране для контролю обчислювальної вартості експериментів. При більших кластерах матриця суміжності зростає квадратично, що збільшує розмірність вектора ознак та вимоги до пам'яті. Як наслідок, модель не може бути безпосередньо застосована до кластерів з більшою кількістю вузлів без перенавчання на відповідних даних. У реальних виробничих кластерах кількість нод часто значно перевищує 15, що обмежує безпосередню практичну застосовність системи у її поточному вигляді.

Незважаючи на перелічені обмеження, проведене експериментальне дослідження дозволяє сформулювати ряд обґрунтованих висновків щодо ефективності реалізованих моделей та процедур постобробки. Зокрема, доведено принципову необхідність калібрування ймовірностей при роботі зі збагаченими тренувальними вибірками, продемонстровано ефективність ізотонічного калібрування як універсального методу, обґрунтовано вибір методу Юдена як основного методу пошуку оптимального порогу, визначено CatBoost та E3_CB_Nureg як найкращі моделі за сукупністю метрик. Усунення ж зазначених обмежень визначає основні напрями подальшого розвитку системи у майбутньому.

Висновки до розділу 4

У четвертому розділі представлено результати експериментальних досліджень реалізованих моделей машинного навчання. Усі експерименти проводились на фіксованій тестовій вибірці обсягом 20 000 прикладів з модельним розподілом класів. Експериментально продемонстровано критичну важливість процедури калібрування ймовірностей. Після ізотонічного калібрування ESE для

всіх моделей знижується більш ніж у 60 разів, що відповідає коректному узгодженню прогнозованих ймовірностей з реальними частотами класів.

Виконано порівняльний аналіз трьох методів вибору порогу класифікації. Метод Юдена забезпечує оптимальний загальний баланс між чутливістю та специфічністю і обрано як рекомендований за замовчуванням. Оптимальні пороги Юдена для всіх моделей знаходяться у діапазоні 0.0899-0.1137, що практично співпадає з модельною часткою позитивного класу. Метод MaxF1 видає вищі пороги і дещо консервативніші прогнози. Метод High-Recall забезпечує Recall понад 0.80 для всіх моделей ціною низького Precision.

Комплексний порівняльний аналіз шести моделей виявив, що найкращу якість за сукупністю метрик демонструє CatBoost. Серед ансамблевих архітектур найкращим є E3_CB_Nureg. Жоден з ансамблів у даній конфігурації не перевершує найкращу базову модель CatBoost, що пояснюється високою індивідуальною якістю CatBoost відносно інших компонентів.

ВИСНОВКИ

У дипломній роботі виконано задачу розробки програмної системи бінарної класифікації конфігурацій мікросервісного кластера за станом проблеми розщеплення кластера на основі методів машинного навчання. У результаті виконаної роботи отримано наступні підсумкові результати.

Проведено аналітичний огляд предметної області. Було розглянуто особливості розподілених систем та мікросервісної архітектури, обґрунтовано актуальність задачі прогнозування ПРК у мікросервісних кластерах, проаналізовано існуючі алгоритмічні підходи до її запобігання та їх обмеження. Виконано порівняльний огляд дванадцяти методів машинного навчання, на основі якого обрано три найбільш придатні для задачі алгоритми, а саме Random Forest, Gradient Boosting та CatBoost.

Сформульовано формальну постановку задачі бінарної класифікації, розроблено методику генерації синтетичних даних за моделлю випадкових графів Ердьош-Реньї з параметром $p_{\text{link}} = 0.7$ та модельною часткою позитивних прикладів 9%. Запропоновано padding стратегію перетворення матриць суміжності змінного розміру у вектор фіксованої розмірності 240. Введено чітке розбиття даних на чотири незалежні вибірки з обґрунтуванням різних розподілів класів у кожній з них.

Реалізовано повну програмну систему мовою Python з використанням бібліотек scikit-learn, catboost, NumPy та фреймворку Django. Реалізовано три базові моделі машинного навчання з детально документованими гіперпараметрами та коректним балансуванням класів через константу NATURAL_SPW. Реалізовано три ансамблеві архітектури з різними стратегіями диверсифікації. Розроблено процедуру ізотонічного калібрування ймовірностей та три методи пошуку оптимального порогу класифікації. Реалізовано веб-інтерфейс системи з інтерактивним графічним редактором топології кластера.

Проведено експериментальне дослідження якості реалізованих моделей на фіксованій тестовій вибірці обсягом 20000 прикладів. Експериментально продемонстровано фундаментальну важливість калібрування ймовірностей. ЕСЕ для CatBoost знижується з 0.6401 до 0.0030, а Brier score з 0.5198 до 0.0683. Найвищу якість за сукупністю метрик демонструє CatBoost з показниками ROC-AUC = 0.7961, PR-AUC = 0.3588, статистика Юдена = 0.4287, F1-score при MaxF1 = 0.3607. Серед ансамблевих архітектур найкращою виявилась E3_CB_Hyper зі статистикою Юдена 0.4100 та ROC-AUC 0.7849. Виявлено, що в даній конфігурації жоден ансамбль не перевершує найкращу базову модель за всіма метриками, що пояснюється високою індивідуальною якістю CatBoost і свідчить про специфіку конкретної задачі.

У роботі перелічено обмеження проведеного дослідження, що визначають область коректної інтерпретації отриманих результатів. Усі описані обмеження є природними для початкового етапу досліджень у даному напрямку та визначають конкретні задачі подальшого розвитку.

Практичне значення отриманих результатів полягає у наступному. Розроблено повний програмний комплекс із публічним вихідним кодом [27], що включає шість навчених і відкаліброваних моделей машинного навчання, веб-інтерфейс для інтерактивного використання та повну документацію з відтворення експериментів. Система може застосовуватися архітекторами розподілених систем для порівняльної оцінки альтернативних топологій кластера на етапі проектування. Розроблені компоненти можуть бути перевикористані у суміжних задачах бінарної класифікації з дисбалансом класів за межами безпосередньої тематики ПРК.

Основними напрямками розвитку системи є наступні. По-перше, розширення на динамічні топології з часовою еволюцією, що дозволить переформулювати задачу як справжнє прогнозування передвісників ПРК на основі послідовностей станів кластера. По-друге, перехід до задач з частковою спостережуваністю, де модель отримує неповну інформацію про матрицю суміжності і має оцінювати ймовірність ПРК на основі непрямих сигналів. По-третє, валідація системи на

реальних логах виробничих кластерів для підтвердження переносимості результатів з синтетичних даних. По-четверте, повторення експериментів з різними значеннями `random_seed` для надійної статистичної оцінки відмінностей між моделями та оцінки довірчих інтервалів метрик якості.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Söylemez M., Tekinerdogan B., Kolukısa Tarhan A. Challenges and Solution Directions of Microservice Architectures: A Systematic Literature Review. *Applied Sciences*. 2022. Vol. 12. P. 5507. DOI: <https://doi.org/10.3390/app12115507>.
2. Zheludkov M., Bhuiyan S. The Apache Ignite Book. [S. l.] : Leanpub, 2019. URL: <https://leanpub.com/apacheignite>
3. Міністерство енергетики України. Відбулося засідання Штабу з ліквідації наслідків надзвичайних ситуацій у столиці та на Київщині. URL: <https://mev.gov.ua/novyna/vidbulosya-zasidannya-shtabu-z-likvidatsiyi-naslidkiv-nadzvychnykh-sytuatsiy-u-stolytsi-ta>
4. Ongaro D., Ousterhout J. In search of an understandable consensus algorithm (extended version). 2014. URL: <https://raft.github.io/raft.pdf>
5. Shapiro M., Preguiça N., Baquero C., Zawirski M. Conflict-free replicated data types. *Symposium on Self-Stabilizing Systems*. Springer, 2011. P. 386–400. URL: <https://hal.inria.fr/inria-00609399/document>
6. Whittaker M., Charapko A., Hellerstein J. M., Howard H., Stoica I. Read-Write Quorum Systems Made Practical. *8th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC'21)*. New York : ACM, 2021. DOI: <https://doi.org/10.1145/3447865.3457962>.
7. Lakshman A., Malik P. Cassandra: a decentralized structured storage system. *ACM SIGOPS Operating Systems Review*. 2010. Vol. 44, No. 2. P. 35–40. URL: https://cassandra.apache.org/_/cassandra_en.pdf
8. Building Scalable, Resilient, and Highly Available Applications in the AWS Cloud : AWS Whitepaper. Amazon Web Services. URL: <https://aws.amazon.com/whitepapers/>
9. Louppe G. Understanding random forest. From theory to practice. *arXiv*. 2014. URL: <https://arxiv.org/pdf/1407.7502>

10. Maulud D., Abdulazeez A. M. A Review on Linear Regression Comprehensive in Machine Learning. *Journal of Applied Science and Technology Trends*. 2020. Vol. 1, No. 2. P. 140–147.
11. Mezquita Y., Alonso R. S., Casado-Vara R., Prieto J., Corchado J. M. A Review of k-NN Algorithm Based on Classical and Quantum Machine Learning. *Distributed Computing and Artificial Intelligence, Special Sessions, 17th International Conference. DCAI 2020. Advances in Intelligent Systems and Computing*, vol. 1242. Springer, Cham, 2021. DOI: https://doi.org/10.1007/978-3-030-53829-3_20.
12. Narayan Y. Comparative analysis of SVM and Naive Bayes classifier for the SEMG signal classification. *Materials Today: Proceedings*. Elsevier, 2021.
13. Feng D. C., Liu Z. T., Wang X. D., Chen Y., Chang J. Q., Wei D. F., Jiang Z. M. Machine learning-based compressive strength prediction for concrete: An adaptive boosting approach. *Construction and Building Materials*. 2020. Vol. 230. P. 117000.
14. Brownlee J. A Gentle Introduction to the Gradient Boosting Algorithm for Machine Learning. *Machine Learning Mastery*. 2020. URL: <https://machinelearningmastery.com/gentle-introduction-gradient-boosting-algorithm-machine-learning/>
15. Yan J., Xu Y., Cheng Q. et al. LightGBM: accelerated genomically designed crop breeding through ensemble learning. *Genome Biology*. 2021. Vol. 22, No. 271. DOI: <https://doi.org/10.1186/s13059-021-02492-y>.
16. Hancock J. T., Khoshgoftaar T. M. CatBoost for big data: an interdisciplinary review. *Journal of Big Data*. 2020. Vol. 7. P. 94. DOI: <https://doi.org/10.1186/s40537-020-00369-8>.
17. Mathew A., Amudha P., Sivakumari S. Deep Learning Techniques: An Overview. *Advanced Machine Learning Technologies and Applications. AMLTA 2020. Advances in Intelligent Systems and Computing*, vol. 1141. Springer, Singapore, 2021. DOI: https://doi.org/10.1007/978-981-15-3383-9_54.
18. Van Rossum G., Drake F. L. Python 3 Reference Manual. Scotts Valley : CreateSpace, 2009. 150 p.

19. Pedregosa F., Varoquaux G., Gramfort A. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011. Vol. 12. P. 2825–2830.
20. Harris C. R., Millman K. J., van der Walt S. J. et al. Array programming with NumPy. *Nature*. 2020. Vol. 585. P. 357–362. DOI: <https://doi.org/10.1038/s41586-020-2649-2>.
21. Django Software Foundation. Django Web Framework. 2026. URL: <https://www.djangoproject.com/>
22. Friedman J. H. Stochastic gradient boosting. *Computational Statistics & Data Analysis*. 2002. Vol. 38, No. 4. P. 367–378. DOI: [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
23. Zadrozny B., Elkan C. Transforming Classifier Scores into Accurate Multiclass Probability Estimates. *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York : ACM, 2002. P. 694–699.
24. Niculescu-Mizil A., Caruana R. Predicting Good Probabilities with Supervised Learning. *Proceedings of the 22nd International Conference on Machine Learning*. New York : ACM, 2005. P. 625–632.
25. Youden W. J. Index for rating diagnostic tests. *Cancer*. 1950. Vol. 3, No. 1. P. 32–35.
26. Elkan C. The foundations of cost-sensitive learning. *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*. 2001. Vol. 17, No. 1. P. 973–978.
27. PSnik-Kostiantyn. SplitBrainDetector. 2025. URL: <https://github.com/PSnik-Kostiantyn/SplitBrainDetector>
28. Сінько Д. П., Сінько К. Д. Аналіз застосовності методів машинного навчання у вирішенні задачі прогнозування реалізації факторів розщеплення кластера. *Електронне моделювання*. 2025. Т. 47, № 1. С. 22–39. DOI: <https://doi.org/10.15407/emodel.47.01.022>.

29. Сінько Д. П., Сінько К. Д. Застосування методів машинного навчання у прогнозуванні факторів, що вказують на потенційне розщеплення кластера. *Електронне моделювання*. 2025. Т. 47, № 6. С. 58–68. DOI: <https://doi.org/10.15407/emodel.47.06.058>.

30. Сінько Д. П., Сінько К. Д. Вирішення проблеми розмірності у задачі прогнозування проблеми розщеплення кластера. *Кібербезпека енергетики : збірник матеріалів науково-практичної конференції* (м. Київ, 28 травня 2025 р.). Київ : Інститут проблем моделювання в енергетиці ім. Г. Є. Пухова НАН України, 2025. С. 65–66.

ДОДАТОК А. Код класу EnsembleModel

class EnsembleModel:

```
def __init__(self, models: list, weights=None, name: str = "Ensemble"):
    self.models = models
    self.weights = np.array(weights or [1.0] * len(models), dtype=float)
    self.weights /= self.weights.sum()
    self.name = name
    self._cal_models = None

def predict_proba_raw(self, X: np.ndarray) -> np.ndarray:
    p = np.zeros(len(X))
    for m, w in zip(self.models, self.weights):
        p += w * m.predict_proba(X[:, 1])
    return p

def predict_proba_calibrated(self, X: np.ndarray) -> np.ndarray:
    if self._cal_models is None:
        raise RuntimeError("Not calibrated. Call calibrate() first.")
    p = np.zeros(len(X))
    for m, w in zip(self._cal_models, self.weights):
        p += w * m.predict_proba(X[:, 1])
    return np.clip(p, 0.0, 1.0)

def predict(self, nodes: list, matrix, use_calibrated: bool = True) -> float:
    x = preprocess(nodes, matrix).reshape(1, -1)
    if use_calibrated and self._cal_models is not None:
        return float(self.predict_proba_calibrated(x)[0])
    return float(self.predict_proba_raw(x)[0])

def calibrate(self, X_cal: np.ndarray, y_cal: np.ndarray,
              method: str = "isotonic"):
    self._cal_models = []
    for i, m in enumerate(self.models):
        cal = CalibratedClassifierCV(estimator=m, method=method, cv="prefit")
        cal.fit(X_cal, y_cal)
        self._cal_models.append(cal)
        print(f" [{self.name}] model {i + 1}/{len(self.models)} calibrated")

def save(self, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "wb") as f:
        pickle.dump(self, f)
    print(f" Saved: {path} ({path.stat().st_size / 1e6:.1f} MB)")

@staticmethod
def load(path) -> "EnsembleModel":
    with open(Path(path), "rb") as f:
        return _FixedUnpickler(f).load()
```

```

class _FixedUnpickler(pickle.Unpickler):

    def find_class(self, module, name):
        if name == "EnsembleModel":
            return EnsembleModel
        return super().find_class(module, name)

ENSEMBLE_DIR = Path("models/ensembles")
PATHS = {
    "e1": ENSEMBLE_DIR / "ensemble1_mixed.pkl",
    "e2": ENSEMBLE_DIR / "ensemble2_cb_bias.pkl",
    "e3": ENSEMBLE_DIR / "ensemble3_cb_hyper.pkl",
    "e1_cal": ENSEMBLE_DIR / "ensemble1_mixed_cal.pkl",
    "e2_cal": ENSEMBLE_DIR / "ensemble2_cb_bias_cal.pkl",
    "e3_cal": ENSEMBLE_DIR / "ensemble3_cb_hyper_cal.pkl",
}
CONFIGS = {
    "full": {"n_base": 800_000, "val_size": 20_000, "cal_size": 16_000},
    "quick": {"n_base": 5_000, "val_size": 500, "cal_size": 500},
}

NATURAL_SPW = (1 - 0.09) / 0.09 + 25

SB_RATIO = 0.35

_cache: dict = {}

def _gen_balanced(n_normal, n_sb, tag=""):
    X, y = [], []
    for _ in tqdm(range(n_normal), desc=f"{tag} normal", leave=False):
        nodes, matrix = generate_cluster()
        while isClusterDead(nodes, matrix):
            nodes, matrix = generate_cluster()
        X.append(preprocess(nodes, matrix))
        y.append(int(isSplitBrain(nodes, matrix)))
    for _ in tqdm(range(n_sb), desc=f"{tag} SB forced", leave=False):
        nodes, matrix = generate_cluster()
        while not isSplitBrain(nodes, matrix):
            nodes, matrix = generate_cluster()
        X.append(preprocess(nodes, matrix))
        y.append(1)
    idx = np.random.permutation(len(X))
    return np.array(X)[idx], np.array(y)[idx]

def _gen_natural(n, tag=""):
    X, y = [], []
    for _ in tqdm(range(n), desc=f"{tag} natural", leave=False):
        nodes, matrix = generate_cluster()
        while isClusterDead(nodes, matrix):

```

```

        nodes, matrix = generate_cluster()
        X.append(preprocess(nodes, matrix))
        y.append(int(isSplitBrain(nodes, matrix)))
    return np.array(X), np.array(y)

def _sb_split(n_total, sb_ratio):
    if sb_ratio <= 0.09:
        return n_total, 0
    n_sb = int((sb_ratio - 0.09) / (1 - 0.09) * n_total)
    n_normal = n_total - n_sb
    return n_normal, n_sb

def train_el(cfg) -> EnsembleModel:
    n, vs = cfg["n_base"], cfg["val_size"]
    n_ok, n_sb = _sb_split(n, SB_RATIO)
    print(f'\n{'=' * 60}')
    print(f" E1: RF + GB + CB (total={n:,}, forced_SB={n_sb:,}, "
          f"target ~ {SB_RATIO * 100:.0f}% SB)")
    print(f'\n{'=' * 60}')

    X, y = _gen_balanced(n_ok, n_sb, "E1")
    pos = int((y == 1).sum());
    neg = int((y == 0).sum())
    actual_pct = pos / len(y) * 100
    sw = np.where(y == 1, NATURAL_SPW, 1.0)
    print(f" actual: pos={pos} neg={neg} ( {actual_pct:.1f}% SB) "
          f"NATURAL_SPW={NATURAL_SPW:.2f}")

    print("[E1] RF...")
    rf = RandomForestClassifier(
        n_estimators=400, max_depth=12, min_samples_leaf=4,
        max_features="sqrt", max_samples=0.8,
        class_weight="balanced_subsample",
        oob_score=True, random_state=42, n_jobs=-1,
    )
    rf.fit(X, y)
    print(f" OOB={rf.oob_score_:.4f}")

    print("[E1] GB...")
    gb = GradientBoostingClassifier(
        n_estimators=500, learning_rate=0.05, max_depth=8,
        subsample=0.8, max_features=0.8,
        n_iter_no_change=20, validation_fraction=0.1,
        random_state=42, verbose=0,
    )
    gb.fit(X, y, sample_weight=sw)
    print(f" estimators={gb.n_estimators}")

    print("[E1] CB...")
    X_val, y_val = _gen_natural(vs, "E1-val")
    cb = CatBoostClassifier(

```



```

        iterations=5000, depth=7, learning_rate=0.05,
        loss_function="Logloss", eval_metric="AUC",
        scale_pos_weight=NATURAL_SPW,
        early_stopping_rounds=150,
        grow_policy="Lossguide", min_data_in_leaf=10,
        l2_leaf_reg=5.0, verbose=0, thread_count=-1, random_seed=42,
    )
    cb.fit(X, y, eval_set=(X_val, y_val), use_best_model=True)
    print(f" best_iter={cb.best_iteration_}")

    return EnsembleModel([rf, gb, cb], name="E1_Mixed")

def _cb_ratio(n_total, sb_ratio, val_size, tag) -> CatBoostClassifier:
    n_ok, n_sb = _sb_split(n_total, sb_ratio)
    X, y = _gen_balanced(n_ok, n_sb, tag)
    pos = int((y == 1).sum())
    neg = int((y == 0).sum())
    spw = neg / max(pos, 1)
    actual_pct = pos / len(y) * 100
    print(f" [{tag}] neg={neg} pos={pos} "
          f"({actual_pct:.1f}% SB, target {sb_ratio * 100:.0f}%) spw={spw:.2f}")
    X_val, y_val = _gen_natural(val_size, f'{tag}-val')
    m = CatBoostClassifier(
        iterations=5000, depth=7, learning_rate=0.05,
        loss_function="Logloss", eval_metric="AUC",
        scale_pos_weight=spw,
        early_stopping_rounds=150,
        grow_policy="Lossguide", min_data_in_leaf=10,
        l2_leaf_reg=5.0, verbose=0, thread_count=-1,
        random_seed=int(sb_ratio * 100),
    )
    m.fit(X, y, eval_set=(X_val, y_val), use_best_model=True)
    print(f" [{tag}] best_iter={m.best_iteration_}")
    return m

def train_e2(cfg) -> EnsembleModel:
    n, vs = cfg["n_base"], cfg["val_size"]
    print(f"\n{'=' * 60}")
    print(f" E2: CB-10% + CB-50% + CB-90% (n={n:,})")
    print(f"{'=' * 60}")
    cb10 = _cb_ratio(n, 0.10, vs, "CB-10%SB")
    cb50 = _cb_ratio(n, 0.50, vs, "CB-50%SB")
    cb90 = _cb_ratio(n, 0.90, vs, "CB-90%SB")
    return EnsembleModel([cb10, cb50, cb90], name="E2_CB_Bias")

def train_e3(cfg) -> EnsembleModel:
    n, vs = cfg["n_base"], cfg["val_size"]
    n_ok, n_sb = _sb_split(n, SB_RATIO)
    print(f"\n{'=' * 60}")
    print(f" E3: CB × 3 hyper (total={n:,}, forced_SB={n_sb:,}, ")

```

```

    f'target ~{SB_RATIO * 100:.0f}% SB)")
print(f'{'=' * 60}')

X, y = _gen_balanced(n_ok, n_sb, "E3")
pos = int((y == 1).sum());
neg = int((y == 0).sum())
print(f' actual: pos={pos} neg={neg} ({pos / len(y) * 100:.1f}% SB) "
      f"NATURAL_SPW={NATURAL_SPW:.2f}")
X_val, y_val = _gen_natural(vs, "E3-val")

kw = dict(
    loss_function="Logloss", eval_metric="AUC",
    scale_pos_weight=NATURAL_SPW,
    early_stopping_rounds=150,
    verbose=0, thread_count=-1,
)

print("[E3] CB-A shallow lr=0.10 depth=5...")
ca = CatBoostClassifier(
    iterations=5000, depth=5, learning_rate=0.10,
    l2_leaf_reg=3.0, random_seed=10, **kw)
ca.fit(X, y, eval_set=(X_val, y_val), use_best_model=True)
print(f' best_iter={ca.best_iteration_}')

print("[E3] CB-B deep lr=0.03 depth=9...")
cb = CatBoostClassifier(
    iterations=5000, depth=9, learning_rate=0.03,
    grow_policy="Lossguide", min_data_in_leaf=15,
    l2_leaf_reg=8.0, random_seed=20, **kw)
cb.fit(X, y, eval_set=(X_val, y_val), use_best_model=True)
print(f' best_iter={cb.best_iteration_}')

print("[E3] CB-C balanced high-reg depth=7...")
cc = CatBoostClassifier(
    iterations=5000, depth=7, learning_rate=0.05,
    grow_policy="SymmetricTree", subsample=0.7,
    l2_leaf_reg=12.0, random_seed=30, **kw)
cc.fit(X, y, eval_set=(X_val, y_val), use_best_model=True)
print(f' best_iter={cc.best_iteration_}')

return EnsembleModel([ca, cb, cc], name="E3_CB_Hyper")

def get_ensemble(key: str, calibrated: bool = True) -> EnsembleModel:
    cache_key = f'{key}_{'cal' if calibrated else 'raw'}'
    if cache_key in _cache:
        return _cache[cache_key]

    path = PATHS[f'{key}_{cal' if calibrated else key}']
    if not path.exists():
        fallback = PATHS[key]
        if fallback.exists():
            ens = EnsembleModel.load(fallback)

```

```

        _cache[cache_key] = ens
        return ens
    raise FileNotFoundError(
        f'Ensemble '{key}' not found at {path}.\n'
        f'Run: python ensemble.py"
    )

ens = EnsembleModel.load(path)
_cache[cache_key] = ens
return ens

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--quick", action="store_true",
                        help="Small dataset for quick test")
    parser.add_argument("--skip", nargs="*", default=[],
                        choices=["e1", "e2", "e3"],
                        help="Skip specific ensembles")
    args = parser.parse_args()

    cfg = CONFIGS["quick" if args.quick else "full"]
    ENSEMBLE_DIR.mkdir(parents=True, exist_ok=True)
    t_total = time.time()

    print(f'\nConfig: {cfg}')
    print(f'SB_RATIO={SB_RATIO * 100:.0f}% NATURAL_SPW={NATURAL_SPW:.2f}\n')

    trainers = {"e1": train_e1, "e2": train_e2, "e3": train_e3}
    trained = {}

    for key in ["e1", "e2", "e3"]:
        if key in args.skip:
            print(f'[{key.upper()}] Skipped — loading existing...')
            trained[key] = EnsembleModel.load(PATHS[key])
        else:
            t0 = time.time()
            trained[key] = trainers[key](cfg)
            trained[key].save(PATHS[key])
            print(f'[{key.upper()}] Done in {time.time() - t0:.1f}s')

    print(f'\n[Cal] Generating calibration set "
          f'({cfg["cal_size"]} samples, natural distribution)...')
    X_cal, y_cal = _gen_natural(cfg["cal_size"], "[Cal]")
    print(f'[Cal] SB rate={y_cal.mean():.3f} '
          f'({int(y_cal.sum())} SB / {len(y_cal)})')

    for key, ens in trained.items():
        print(f'\n[Cal] Calibrating {key.upper()}...')
        ens.calibrate(X_cal, y_cal)
        ens.save(PATHS[f'{key}_cal'])
        print(f' [{key.upper()}] calibrated and saved.')

```

```
elapsed = time.time() - t_total
h, rem = divmod(int(elapsed), 3600)
m, s = divmod(rem, 60)
print(f'\n{'─' * 60}')
print(f' Complete in {h:02d}:{m:02d}:{s:02d}')
print(f' Files: {ENSEMBLE_DIR.resolve()}/')
print(f{'─' * 60}\n")
```

ДОДАТОК Б. Апробація результатів

Б.1 Стаття у журналі «Електронне моделювання», № 47(1), 2025 р.

Бібліографічний опис:

1. Сінько Д.П., Сінько К.Д. Аналіз застосовності методів машинного навчання у вирішенні задачі прогнозування реалізації факторів розщеплення кластера. Електронне моделювання. 2025. Т. 47, № 1. С. 22–39. DOI: <https://doi.org/10.15407/emodel.47.01.022>.

Посилання на електронну версію: <https://www.emodel.org.ua/en/archive/2025/47-1/47-1-2>

У статті проаналізовано застосовність методів машинного навчання для прогнозування факторів, що вказують на реалізацію розщеплення кластера у мікросервісних системах. Розглянуто алгоритми Random Forest, Gradient Boosting, CatBoost. Сформульовано підхід до побудови моделей на основі топологічних характеристик кластера.



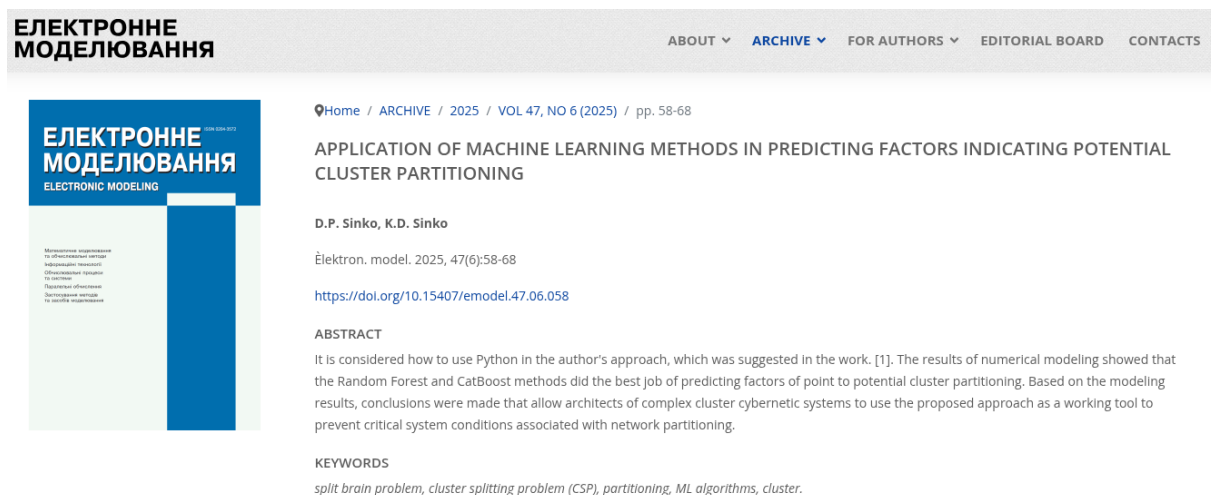
Б.2 Стаття у журналі «Електронне моделювання», № 47(6), 2025 р.

Бібліографічний опис:

2. Сінько Д.П., Сінько К.Д. Застосування методів машинного навчання у прогнозуванні факторів, що вказують на потенційне розщеплення кластера. Електронне моделювання. 2025. Т. 47, № 6. С. 58–68. DOI: <https://doi.org/10.15407/emodel.47.06.058>.

Посилання на електронну версію: <https://www.emodel.org.ua/en/archive/2025/47-6/47-6-4>

У статті описано застосування мови Python для реалізації авторського підходу до прогнозування факторів розщеплення кластера. Результати числового моделювання показали, що методи Random Forest та CatBoost забезпечують найкраще прогнозування зазначених факторів. На основі отриманих результатів сформульовано рекомендації архітекторам складних кібернетичних кластерних систем щодо використання запропонованого підходу для запобігання критичним станам, пов'язаним з мережевим розщепленням.



Б.3 Тези доповіді на конференції КБЕ-2025

Бібліографічний опис:

3. Сінько Д.П., Сінько К.Д. Вирішення проблеми розмірності у задачі прогнозування проблеми розщеплення кластера. Кібербезпека енергетики: збірник матеріалів науково-практичної конференції (м. Київ,

28 травня 2025 р.). Київ: Інститут проблем моделювання в енергетиці ім. Г.Є. Пухова НАН України, 2025. С. 65-66.

Посилання на електронну версію збірника: <https://ipme.kiev.ua/wp-content/uploads/2025/05/%D0%9C%D0%B0%D1%82%D0%B5%D1%80%D1%96%D0%B0%D0%BB%D0%B8-%D0%9A%D0%91%D0%95-2025-1.pdf#page=65>

В тезах розглядається прогнозування проблеми розщеплення кластера (ПРК). Основна увага приділяється викликам, пов'язаним із змінною розмірністю кластерів, що ускладнює подачу даних на вхід моделям машинного навчання. Проаналізовано сучасні методи уніфікації структур графів, зокрема техніки padding і маскування. Окремо розглянуто спектральні підходи двовимірне перетворення Лапласа та графове перетворення Фур'є. Робота підкреслює переваги та обмеження кожного з методів та окреслює напрямки для побудови розмірно-універсальних моделей.

Д.П. Сінько, К.Д. Сінько

ВИРІШЕННЯ ПРОБЛЕМИ РОЗМІРНОСТІ У ЗАДАЧІ ПРОГНОЗУВАННЯ ПРОБЛЕМИ РОЗЩЕПЛЕННЯ КЛАСТЕРА

Анотація

В тезах розглядається прогнозування проблеми розщеплення кластера (ПРК). Основна увага приділяється викликам, пов'язаним із змінною розмірністю кластерів, що ускладнює подачу даних на вхід моделям машинного навчання. Проаналізовано сучасні методи уніфікації структур графів, зокрема техніки padding і маскування. Окремо розглянуто спектральні підходи двовимірне перетворення Лапласа та графове перетворення Фур'є. Робота підкреслює переваги та обмеження кожного з методів та окреслює напрямки для побудови розмірно-універсальних моделей.

Вступ

У сучасній архітектурі розподілених інформаційних систем майже неможливо обійтися без застосування мікросервісного підходу. Водночас мікросервісна архітектура супроводжується ризиками, зокрема виникненням проблеми розщеплення кластера (далі ПРК) критичного стану кластерних систем, спричиненого їх розподілом на ізольовані частини, в англійській літературі це явище відоме як *splitbrainproblem* [1]. В контексті мілітарних загроз в Україні [2] та проблем зі світлом та зв'язком у світі [3] дане питання набуває надзвичайної актуальності.

У попередній роботі [4] було запропоновано використання методів та алгоритмів машинного навчання для прогнозування ПРК. Розглянуто метод забезпечення консистентності в кластерах з реплікацією типу *singleleader* [5], за якого одна з нод виконує функції лідера, а у випадку розділення кластера решта його частин орієнтуються на рішення цієї ноди. Залишається актуальною задача створення розмірно-універсальної моделі, яка б могла приймати кластери різного розміру та прогнозувати ймовірність ПРК. Проаналізуємо підходи з відкритих джерел.

Виклад основного матеріалу

У роботі [6] представлення кластеру відбувається у формі матриці суміжності та пропонується підхід перетворення вхідних даних змінної розмірності до фіксованого формату шляхом доповнення матриці

ДОДАТОК В. Презентація



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці

Розмірно-універсальні моделі резильєнтності кластерних систем щодо проблеми розщеплення кластера

виконав: студент групи ТВ-23 Сінько Костянтин Дмитрович

керівник: асист. Макарчук Андрій Валентинович

2026

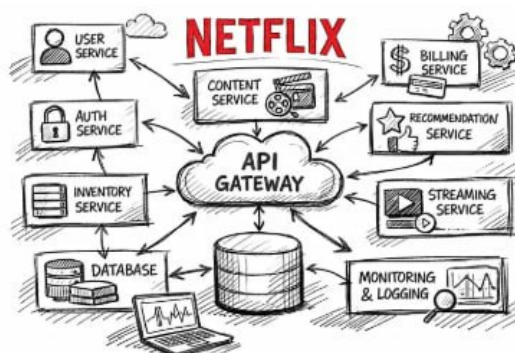
Актуальність теми

Мікросервісна архітектура є одним з найпопулярніших виборів архітектури для програмного забезпечення. Її використовують зокрема Netflix, Amazon, Monobank, Prom

Мікросервісна архітектура (MSA): коли потрібна бізнесу

Поради #CloudArchiveStorage #ПрограмнеЗабезпечення #Сервіс

Microservices vs Monolithic Architecture in Banking Systems



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

2

Актуальність теми

Такий тип архітектури особливо вразливий перед сучасними викликами у світі та Україні. Збої у мережі призводять до критичних наслідків.

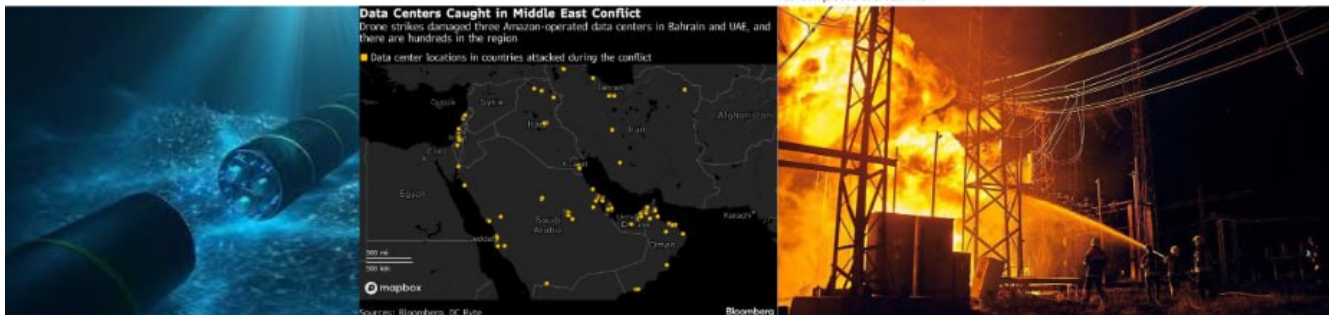
Postgres DB Split brain incident led to data loss - postmortem.

[Closed](#) Issue created Jul 12, 2016 by Pablo Carranza [GitLab]

This morning (Europe time) we had an issue that led to data loss when both databases, DB4 and DB5, acted as master for roughly 30 minutes.

Оперативна ситуація в енергосистемі України станом на ранок 27 травня

Внаслідок бойових дій та обстрілів енергетичних об'єктів частина споживачів у Дніпропетровській, Донецькій, Запорізькій, Сумській та Харківській областях тимчасово залишаються без електропостачання.

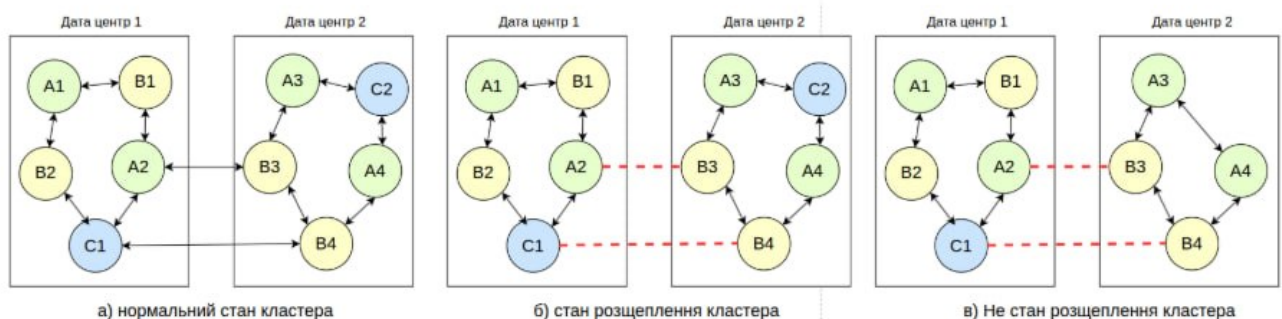


Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

3

Постановка задачі

Проблема розщеплення кластера (ПРК)- це ситуація в розподіленій системі, коли через втрату зв'язку між вузлами кілька частин кластера одночасно вважають себе головними та незалежно роблять записи, що призводить до неконсистентності даних.



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

4

Постановка задачі

1. Огляд існуючих підходів до запобігання ПРК та методів машинного навчання
2. Розробка методики генерації синтетичних навчальних даних
3. Реалізація трьох базових моделей машинного навчання та трьох ансамблевих
4. Калібрування ймовірностей та трьох методів пошуку оптимального порогу
5. Експериментальне дослідження та розробка веб-інтерфейсу

Матриця зв'язків:

	A1	A2	A3	B1	B2	C1	C2	C3
A1	0	0	0	0	0	1	0	0
A2	0	0	0	0	0	0	0	0
A3	0	0	0	1	0	0	1	0
B1	0	1	1	0	0	1	0	0
B2	0	0	0	0	0	0	0	1
C1	1	0	1	1	0	0	1	0
C2	0	0	1	0	0	0	0	1
C3	0	0	0	0	0	0	1	0

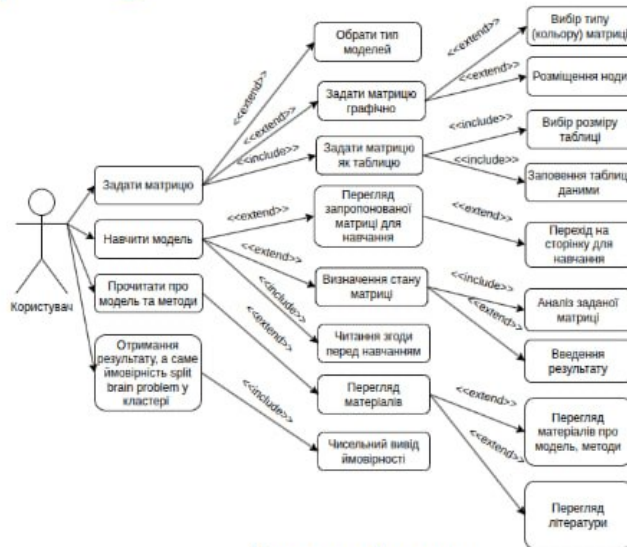


Аналіз предметної області

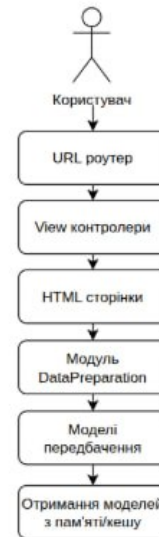
Рішення	Підхід	Обмеження
Raft / Paxos	Консенсус кворуму	Реактивний, після ПРК
CRDT	Eventual consistency без консенсусу	Обмеженість використання
Системи від AWS	Автовідновлення	Прив'язка до екосистеми



Проектування системи



Use case діаграма



Діаграма потоків обробки запиту



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

7

Реалізація системи



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

8

Реалізація системи

Матриця зв'язків:

	A1	A2	A3	B1	B2	D1	D2	C3	C4
A1	0	0	0	0	0	0	1	1	0
A2	0	0	0	0	0	1	1	0	0
A3	0	0	0	1	0	0	0	1	0
B1	0	0	1	0	0	1	0	0	0
B2	0	0	0	0	0	1	0	0	0
D1	0	0	0	0	1	0	1	1	0
D2	0	1	0	0	0	1	0	0	0
C3	1	0	1	0	0	1	0	0	0
C4	0	0	0	0	1	1	0	0	0

Найкращий

Random Forest: 40.86%
Gradient Boosting: **61.93%**
CatBoost: **73.93%**

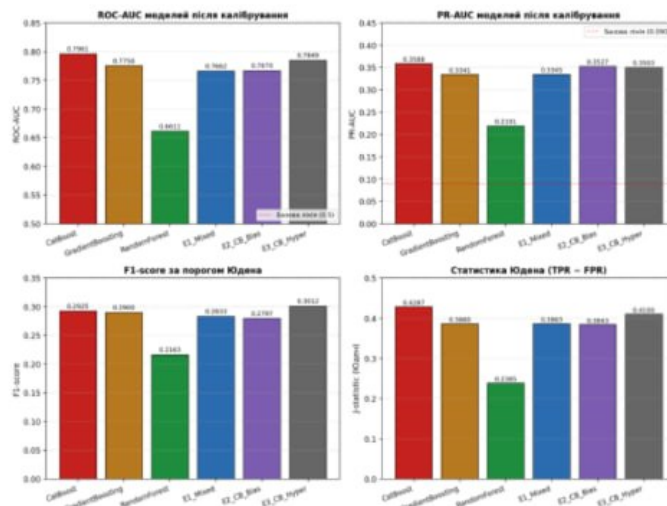


Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

9

Тестування системи

Модель	ROC-AUC	Recall	FNR	ECE
CatBoost	0.797	0.778	0.222	0.003
Gradient Boosting	0.775	0.691	0.309	0.005
Random Forest	0.661	0.739	0.261	0.004
E3 (CB Hyper)	0.785	0.703	0.297	0.006
E2 (CB Bias)	0.767	0.728	0.272	0.008
E1 (Mixed)	0.766	0.720	0.280	0.014



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

10

Впровадження та супровід



The screenshot shows the GitHub repository for **SplitBrainDetector** by P.Snik-Kostianyn. The repository is public and has a README file. The README describes the project as a binary classification system for assessing microservice cluster configurations by split brain probability using machine learning methods. It mentions that SplitBrainDetector is a research-grade machine learning system that evaluates microservice cluster topologies for their susceptibility to the split brain problem (network partition that results in two or more isolated subclusters operating independently). It combines six trained ML models — three base classifiers and three ensemble architectures — with isotonic probability calibration and an interactive web interface for cluster topology design.

The right side of the screenshot shows the cover of the journal **ЕЛЕКТРОННЕ МОДЕЛЮВАННЯ** (Electronic Modeling). The cover features the title in Ukrainian and English, the author's name (D.P. Siniko, K.D. Siniko), the journal name, and the volume/issue information (2025, VOL. 47, NO. 6 (2025), pp. 58-65). The article title is **APPLICATION OF MACHINE LEARNING METHODS IN PREDICTING FACTORS II CLUSTER PARTITIONING**. The abstract mentions the use of Python in the author's approach, which was suggested in the work [1]. The results of the Random Forest and CatBoost methods did the best job of predicting factors of point to potential cluster partitioning. The keywords are: split brain problem, cluster splitting problem (CSP), partitioning, ML algorithms, cluster.

НАУКОВО-ПРАКТИЧНА КОНФЕРЕНЦІЯ
«КІБЕРБЕЗПЕКА ЕНЕРГЕТИКИ»



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

11

Висновки

1. Проведено огляд існуючих підходів та методів машинного навчання
2. Розроблено методику генерації синтетичних даних та постановку задачі
3. Реалізовано три базові та три ансамблеві моделі
4. Реалізовано калібрування ймовірностей та методи вибору порогу
5. Проведено експериментальні дослідження та розроблено веб-інтерфейс



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

12



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

National Technical University of Ukraine
"Igor Sikorsky Kyiv Polytechnic Institute"